

MI FB API

Version 3.0

© 2022 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	07/27/2021
	- 修改图 1-1，图形层支持叠加到 SCL PATH，并新增相关说明 1.5 章节芯片差异部分新增图形层叠加到 SCL PATH 在不同芯片的支持情况 1.7 章节对新增 INI 配置参数做相关介绍	08/08/2021
	新增 PROCFS 介绍	08/25/2021
	改变 HW_LAYER_ID 的定义，并重构芯片差异表格	12/17/2021
	新增 Mochi 芯片差异部分	02/16/2022
	新增 Maruko 芯片差异部分	03/21/2022
	新增 API "FBIO_WAITFORVSYNC"支持	04/11/2022

TABLE OF CONTENTS

REVISION HISTORY.....	i
TABLE OF CONTENTS.....	ii
1. 概述.....	1
1.1. 模块说明	1
1.2. 基本结构	1
1.3. 功能介绍	2
1.4. 应用场景	3
1.5. 芯片差异	3
1.5.1 Taiyaki.....	3
1.5.2 Takoyaki.....	4
1.5.3 Ispahan.....	4
1.5.4 Pudding.....	4
1.5.5 Ikayaki.....	4
1.5.6 Tiramisu.....	4
1.5.7 Muffin.....	4
1.5.8 Mochi.....	5
1.5.9 Maruko.....	5
1.6. 工作原理	6
1.6.1 图形层刷新模式.....	6
1.6.2 虚拟屏幕分辨率、可见区域和显示分辨率的关系.....	7
1.7. 开发流程	7
1.7.1 编译配置.....	7
1.7.2 配置 MI_FB 启动参数.....	8
1.7.3 接口调用.....	10
1.8. 实例介绍	11
2. API 参考.....	16
2.1. 标准功能	18
2.1.1 FBIOGET_VSCREENINFO.....	18
2.1.2 FBIOPUT_VSCREENINFO.....	18
2.1.3 FBIOGET_FSCREENINFO.....	19
2.1.4 FBIOPAN_DISPLAY.....	20
2.1.5 FBIO_WAITFORVSYNC.....	21
2.1.6 FBIOGETCMAP.....	22
2.1.7 FBIOPUTCMAP.....	22
2.2. 扩展功能	23
2.2.1 FBIOGET_SCREEN_LOCATION.....	23
2.2.2 FBIOSET_SCREEN_LOCATION.....	24
2.2.3 FBIOGET_SHOW.....	25
2.2.4 FBIOSET_SHOW.....	26
2.2.5 FBIOGET_GLOBAL_ALPHA.....	26
2.2.6 FBIOSET_GLOBAL_ALPHA.....	27
2.2.7 FBIOGET_COLORKEY.....	28
2.2.8 FBIOSET_COLORKEY.....	28

2.2.9	FBIOSET_DISPLAYLAYER_ATTRIBUTES.....	29
2.2.10	FBIOGET_DISPLAYLAYER_ATTRIBUTES.....	30
2.2.11	FBIOGET_CURSOR_ATTRIBUTE.....	31
2.2.12	FBIOSET_CURSOR_ATTRIBUTE.....	31
2.3.	MI_FB API 参考 32	
2.3.1	MI_FB_Open.....	32
2.3.2	MI_FB_Close.....	33
2.3.3	MI_FB_Mmap.....	34
2.3.4	MI_FB_Munmap.....	35
2.3.5	MI_FB_GetVarScreenInfo.....	36
2.3.6	MI_FB_SetVarScreenInfo.....	37
2.3.7	MI_FB_GetFixScreenInfo.....	38
2.3.8	MI_FB_PanDisplay.....	38
2.3.9	MI_FB_WaitForVsync.....	39
2.3.10	MI_FB_GetScreenLocation.....	40
2.3.11	MI_FB_SetScreenLocation.....	41
2.3.12	MI_FB_GetDisplayLayerAttr.....	41
2.3.13	MI_FB_SetDisplayLayerAttr.....	42
2.3.14	MI_FB_GetCursorAttr.....	43
2.3.15	MI_FB_SetCursorAttr.....	43
2.3.16	MI_FB_GetGlobalAlpha.....	44
2.3.17	MI_FB_SetGlobalAlpha.....	45
2.3.18	MI_FB_GetColorKey.....	46
2.3.19	MI_FB_SetColorKey.....	47
2.3.20	MI_FB_SetCmap.....	48
2.3.21	MI_FB_GetCmap.....	48
2.3.22	MI_FB_GetShow.....	49
2.3.23	MI_FB_SetShow.....	50
3.	数据类型.....	52
3.1.	标准定义 53	
3.1.1	struct fb_bitfield.....	53
3.1.2	struct fb_var_screeninfo.....	53
3.1.3	struct fb_fix_screeninfo.....	55
3.1.4	struct fb_cmap_user.....	57
3.2.	MI_FB 数据类型定义 58	
3.2.1	MI_FB_BitField_t.....	58
3.2.2	MI_FB_VarScreenInfo_t.....	58
3.2.3	MI_FB_FixScreenInfo_t.....	58
3.2.4	MI_FB_ColorFmt_e.....	58
3.2.5	MI_FB_OuputColorSpace_e.....	59
3.2.6	MI_FB_DstDisplayPlane_e.....	60
3.2.7	MI_FB_DisplayLayerAttrMaskBit_e.....	60
3.2.8	MI_FB_CursorAttrMaskBit_e.....	61
3.2.9	MI_FB_DisplayLayerAttr_t.....	62

3.2.10	MI_FB_GlobalAlpha_t.....	63
3.2.11	MI_FB_ColorKey_t.....	64
3.2.12	MI_FB_CursorImage_t.....	65
3.2.13	MI_FB_CursorAttr_t.....	65
3.2.14	MI_FB_Rectangle_t.....	66
3.2.15	MI_FB_Cmap_t.....	67
4.	错误码.....	68
5.	PROCFS 介绍.....	69
5.1.	cat 69	
5.2.	echo 73	

1. 概述

1.1.

模块说明

MI_FB 是一个基于 Linux Framebuff 实现的图形层设备驱动。它将显示缓冲区抽象，屏蔽图像硬件的底层差异，允许上层应用程序在图形模式下直接对显示缓冲区进行读写操作，用户不必关心物理显示缓冲区的具体位置及存放方式，这些都由 MI_FB 模块驱动来完成。在提供 Linux Framebuff 基本功能的基础上，MI_FB 还提供了一些扩展功能，如层间 Alpha、ColorKey、设置显示区域等。

1.2.

基本结构

MI_FB 主要用来管理多个图形层，每一个图形层在芯片能力支持的情况下可以选择叠加到不同的视频处理/输出路径上。

- 1、视频预览叠加路径指图形层可以叠加到视频显示单元上，MI_DISP 子模块用来管理视频显示单元，如果选择叠加到视频显示单元需要首先配置 MI_DISP 子模块，然后再操作 MI_FB 子模块来进行图形层的绘制，最终由视频显示单元将叠加后的视频数据输出到显示设备，视频显示单元决定最终的显示分辨率。
- 2、视频处理叠加路径指图形层可以叠加到视频处理单元上，MI_SCL 子模块用来管理视频处理单元，如果选择叠加到视频处理单元需要首先配置 MI_SCL 子模块，然后再操作 MI_FB 子模块进行图形层的绘制，最终由视频处理单元将叠加后的视频数据输出到视频编码单元，或者输出到视频显示单元进行预览播放。

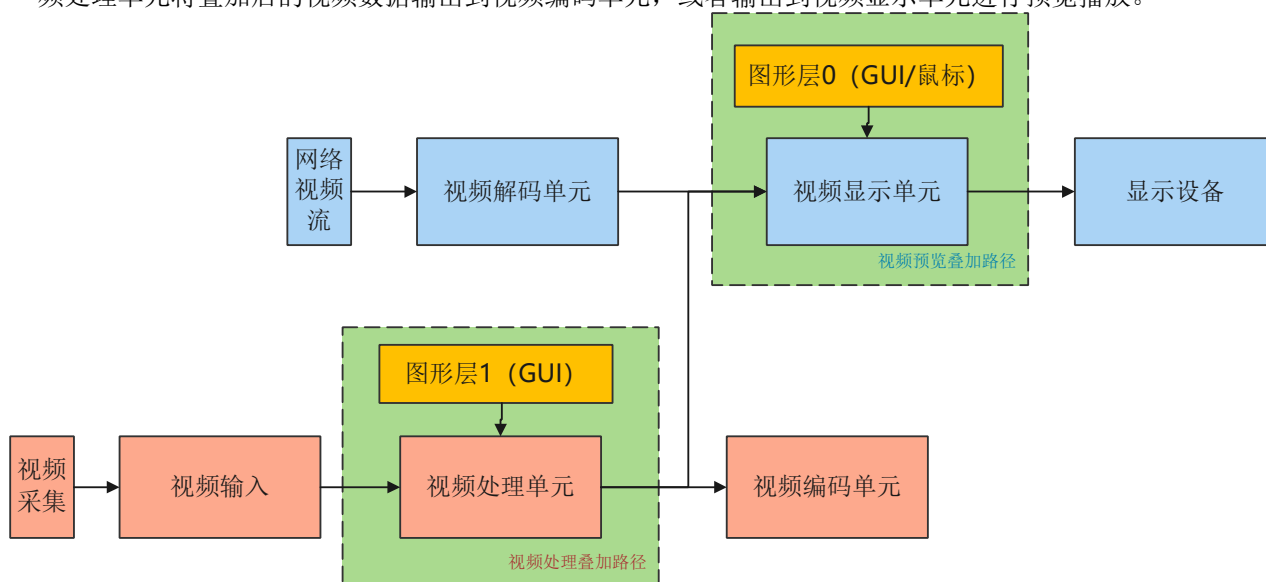


图 1-1 图形层叠加路径示意图

从应用程序的开发角度来看，基于 MI_FB 的图形层绘制有两种方式。

方式 1：应用程序基于 Linux 文件系统使用 MI_FB。

方式 2：应用程序基于 MI_FB 提供的 API 进行开发（提供和方式 1 同样的功能）。

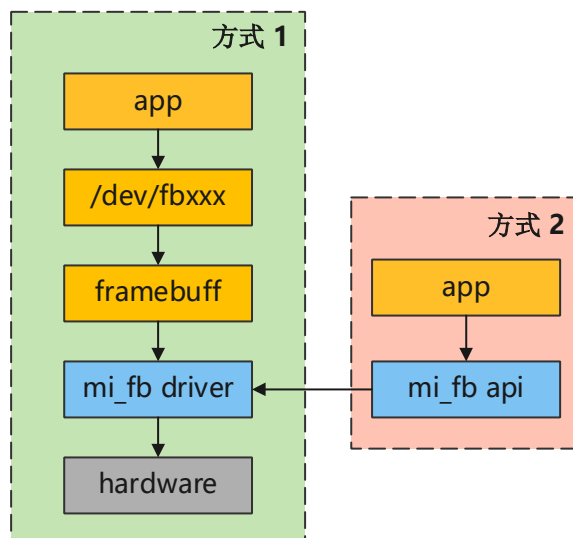


图 1-2 MI_FB 的使用方式

注意：以上两种方式是互斥的，应用程序只能选择其中一种进行开发。

1.3.

功能介绍

MI_FB 支持以下 Linux Framebuff 的标准功能：

- 1、 获取显示设备的固定信息（比如显存大小）
- 2、 将物理显存映射（或解除映射）到虚拟内存空间
- 3、 像操作普通文件一样操作物理显存
- 4、 设置硬件显示分辨率和像素格式
- 5、 从物理显存的任何位置进行读、写、显示等操作
- 6、 支持设置和获取 256 色的调色板

MI_FB 增加以下的扩展功能：

- 1、 设置和获取叠加图形层的 Alpha 值
- 2、 设置和获取叠加图形层的 ColorKey 值
- 3、 设置当前叠加图形层的显示位置（相对于屏幕原点的偏移）
- 4、 设置当前叠加图形层的显示区域（支持画布放大显示）
- 5、 设置和获取当前叠加层的显示状态（显示/隐藏）
- 6、 通过 INI 配置文件管理物理显存大小和叠加图形层的数目
- 7、 动态同步显示设备输出分辨率
- 8、 支持硬件鼠标的一系列操作

MI_FB 不支持的 Linux Framebuff 标准功能：

- 1、 设置和获取控制台对应的 Linux Framebuff
- 2、 获取硬件扫描的实时信息
- 3、 获取硬件相关信息
- 4、 设置硬件同步时序
- 5、 设置硬件同步信号机制

1.4. 应用场景

MI_FB 可应用于以下场景：

1、Pure linux 场景

在 linux 环境下基于 Framebuff 框架的应用程序（例如 MiniGUI），也可以基于 MI_FB 提供的 API 接口进行开发。

2、Pure rtos 场景

在 rtos 环境下，不支援标准 Linux Framebuff 框架，应用程序可以基于 MI_FB 提供的 API 接口进行 UI 的开发（例如使用轻量级的 lvgl 开发 UI）。

3、Dualos 场景

在 dualos 环境下，在 linux 端运行的应用程序可以基于标准 Framebuff 框架，也可以基于 MI_FB API 开发；在 rtos 端运行的应用程序只能基于 MI_FB API 开发。

1.5. 芯片差异

MI_FB 有两种形式的用户态接口，为了让用户行为保持一致，MI_FB 加入了 FB ID 的概念。下图描述了 FB ID 和 Framebuff 设备节点的关系。

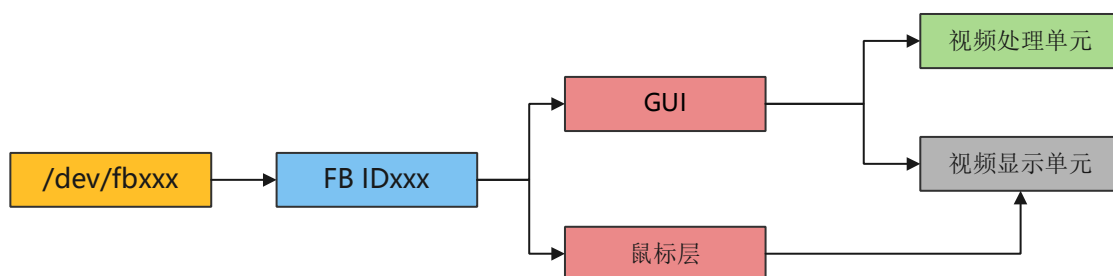


图 1-3 FB ID 和设备节点的关系

MI_FB 定义 ID 号的增长顺序和 Framebuff 的设备节点保持一致，即 /dev/fbxxx 对应 FB IDxxx。

MI_FB 每一个 FB ID 最多可以控制一个 GUI 和一个鼠标层，即应用程序可以使用同一个 FB ID 来同时操作和该 ID 绑定的 GUI 和鼠标层，GUI 可以选择叠加到视频显示单元或者视频处理单元，鼠标层只能叠加到视频显示单元上，GUI 和鼠标层的叠加输出路径在 config.ini 中配置，1.7.2 节会详细描述 INI 的配置方法。

MI_FB 目前可以管理 16 个 FB ID，具体支持的 ID 个数和芯片能力有关，不同芯片的差异如下表所示：

- ※ Layer ID 对应 MI_FB 启动参数中的 FB_HWLAYER_ID 项，配置启动参数时请参照对应芯片表格中的 Layer ID 项进行配置。
- ※ 处于同一视频显示单元同一 PATH 的不同图形层的层级关系自底向上为 GUI -> OSD -> 鼠标。
- ※ Alpha0/1 代表 color format 为 ARGB1555，Alpha bit 为 0/1 时，实际的 Alpha 值。在某些图形层上，只支持设置 Alpha1，不支持设置 Alpha0，此项就标记为 N。

1.5.1 Taiyaki

图形层			Layer ID	颜色格式				I			放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	2	4	8		
视频显示单元	DISP	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N

	PATH0	鼠标	0	N	Y	Y	N	N	N	Y	N	Y
--	-------	----	---	---	---	---	---	---	---	---	---	---

1.5.2 Takoyaki

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频显示单元	DISP	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
	PATH0	鼠标	0	N	Y	Y	N	N	N	Y	N	Y

1.5.3 Ispahan

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频处理单元	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

1.5.4 Pudding

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频处理单元	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

1.5.5 Ikayaki

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频显示单元	DISP	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N

1.5.6 Tiramisu

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频显示单元	DISP	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
	PATH0	鼠标	0	N	Y	Y	N	N	N	Y	N	Y
	DISP	GUI	3	Y	Y	Y	Y	N	N	Y	Y	N
	PATH1	鼠标	2	N	Y	Y	N	N	N	Y	N	Y

视频处理单元	GUI	NA	N	Y	Y	N	N	N	Y	N	Y
--------	-----	----	---	---	---	---	---	---	---	---	---

1.5.7 Muffin

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频显示单元	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		鼠标	0	N	Y	Y	N	N	N	Y	N	Y
		OSD	2	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH1	GUI	9	Y	Y	Y	Y	N	N	Y	Y	N
		鼠标	8	N	Y	Y	N	N	N	Y	N	Y
		OSD	10	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH2	GUI	17	Y	Y	Y	Y	N	N	Y	Y	N
		鼠标	16	N	Y	Y	N	N	N	Y	N	Y
		OSD	18	N	Y	Y	N	N	N	Y	N	Y
视频处理单元	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ Muffin 系列芯片的视频显示单元新增了一个 OSD 图形层，通常作为 MI_RGN 叠加到 DISP 上的默认图形层，如果 MI_FB 需要使用 OSD 图形层，则 MI_RGN 无法叠加 OSD 到 DISP。

1.5.8 Mochi

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		
视频显示单元	DISP PATH0	GUI	1	Y	Y	Y	Y	N	N	Y	Y	N
		鼠标	0	N	Y	Y	N	N	N	Y	N	Y
		OSD	2	N	Y	Y	N	N	N	Y	N	Y
	DISP PATH1	GUI	9	Y	Y	Y	Y	N	N	Y	Y	N
		鼠标	8	N	Y	Y	N	N	N	Y	N	Y
		OSD	10	N	Y	Y	N	N	N	Y	N	Y
视频处理单元	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ Mochi 系列芯片的视频显示单元上的 OSD 图形层通常作为 MI_RGN 叠加到 DISP 上的默认图形层，如果 MI_FB 需要使用 OSD 图形层，则 MI_RGN 无法叠加 OSD 到 DISP。

1.5.9 Maruko

图形层			Layer ID	颜色格式							放大	Alpha0/1
				ARGB8888	ARGB1555	ARGB4444	RGB565	I 2	I 4	I 8		

视频显示单元	DISP PATH0	GUI	2	Y	Y	Y	Y	N	N	Y	N	Y
视频处理单元	GUI		NA	N	Y	Y	N	N	N	Y	N	Y

※ Maruko 系列芯片视频显示单元 GUI 层、视频处理单元 GUI 和 MI_RGN 中的 SCL/DISP GOP 共用同一个硬件，当 MI_RGN 向 SCL/DISP 上叠加 OSD 时，不能使用 FB。反之亦然。

1.6.

工作原理

1.6.1 图形层刷新模式

1、 单 buff 模式

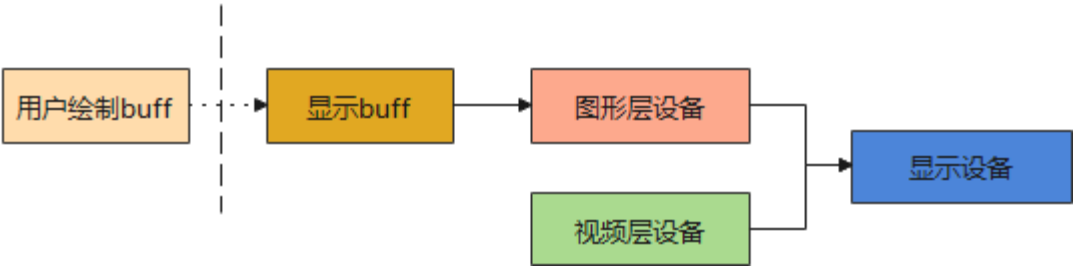


图 1-4 单 buff 刷新模式

在单 buff 模式下，MI_FB 提供一张显示 buff，应用程序可以直接绘制显示 buff，这种方式可以节省内存，但是用户会在显示设备上看到图形的绘制过程。
用户也可以另外申请一块 buff 用于图形绘制，然后在绘制完成后将画面搬移到显示 buff，但是这种方式会看到锯齿。

2、 双 buff 模式

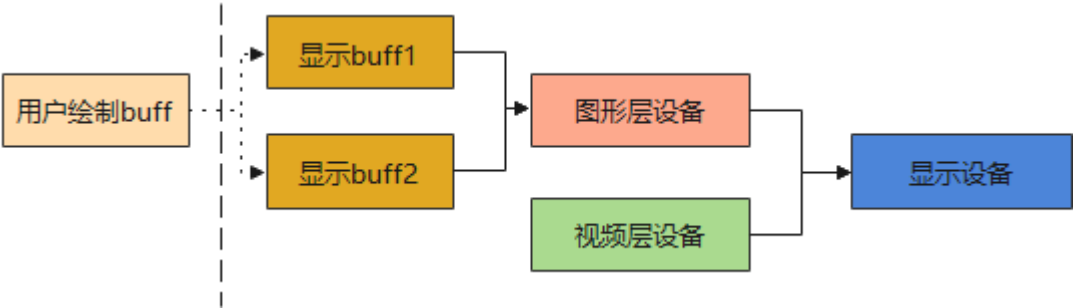


图 1-5 双 buff 刷新模式

在双 buff 模式下，MI_FB 提供两张显示 buff，当图形层设备正在使用显示 buff2 时，用户可以在 buff1 上绘制，当 buff1 绘制完成后，通过设置虚拟屏幕偏移位置使显示 buff 切换到 buff1 的起始位置，反之同理。该模式比较消耗内存，但是显示效果最好。

1.6.2 虚拟屏幕分辨率、可见区域和显示分辨率的关系

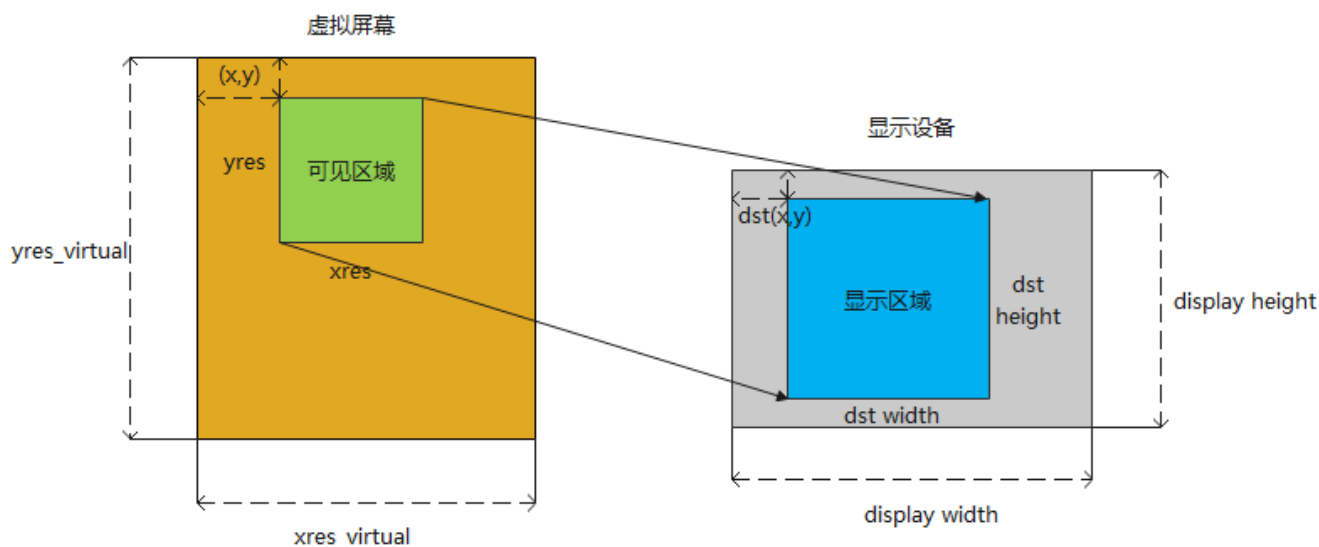


图 1-6 各种分辨率之间的关系

虚拟屏幕的大小可以看作 MI_FB 提供的整个显存的大小，用户可以自己在配置文件中定义可见区域的分辨率（xres,yres），MI_FB 将会根据可见区域的范围调整虚拟屏幕分辨率（xres_virtual, yres_virtual），默认情况下虚拟屏幕分辨率的 $xres_virtual = xoffset + xres$ ，xoffset 是可见区域在虚拟屏幕的水平方向偏移，xres 是可见区域水平方向的 width。

MI_FB 将会根据 xres_virtual 和像素格式以及整个显存的大小调整 yres_virtual；用户可以在（xres_virtual, yres_virtual）范围内随意改变可见区域的（xoffset, yoffset）来切换显示内容，双 buff 的刷新模式也是基于此来实现。

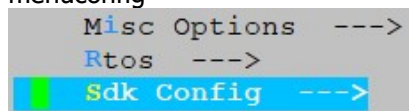
图形层设备将会从可见区域读取内存数据，然后根据用户配置的显示区域属性输出到显示设备，显示区域属性包括显示区域范围和相对显示屏幕原点的偏移位置。

从可见区域到显示区域的过程支持放大，也就是说显示区域的（dst_width, dst_height）可以大于可见区域的（xres, yres），但是不能超过显示设备的分辨率（display width, display height），显示设备的分辨率由 MI_DISP 模块的输出 timing 决定。

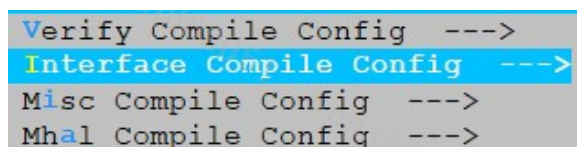
1.7. 开发流程

1.7.1 编译配置

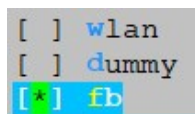
- 1、 进入 alkaid project 根目录，make menuconfig



- 2、 回车键进入 Sdk Config 子选项



3、 回车键进入 Interface Compile Config 子选项



4、 空格键选中 fb 子模块，并重新编译 project

编译完成将在 **project/release** 下生成 **mi_fb.ko** 并同时将 **mi_fb.h**、**mi_fb_datatype.h** release 到该目录，在 **pure linux** 环境下会默认打包进入 **images**，在 **dualos** 环境下默认不打包，需要手动编译并安装。

1.7.2 配置 MI_FB 启动参数

MI_FB 的启动参数在系统配置文件 **config.ini** 中管理，在 **config.ini** 的 **ROOT** section 中定义的 **m_fbdevList** 字段描述了当前平台 **GUI** 或鼠标层参数的入口。

- 1、 **FB_DEVICE** 描述的是叠加到视频显示单元的 **GUI** 层的配置信息；
- 2、 **FB_VIDEO** 和 **FB_BUF_ID0_PATH** 描述的是叠加到视频处理单元的 **GUI** 层的配置信息；
- 3、 **FB_CURSOR** 描述的是鼠标层的配置信息，鼠标层将叠加到 **GUI** 上。

```
[ROOT]
m_fbdevList = {
    FB_DEVICE,\
    FB_CURSOR,\
    FB_VIDEO,\
}

[FB_DEVICE]
FB_HWLAYER_ID = 1;
FB_HWWIN_ID = 0;
FB_HWWIN_FORMAT = 5;
FB_HWLAYER_OUTPUTCOLOR = 0;
FB_WIDTH = 800;
FB_HEIGHT = 480;
FB_TIMING_WIDTH = 1920;
FB_TIMING_HEIGHT = 1080;
FB_MMAP_NAME = E_MMAP_ID_FB;
FB_BUFFER_LEN = 8192
#unit:Kbyte, total size = FB_BUFFER_LEN*1024

[FB_CURSOR]
FB_HWLAYER_ID = 0;
FB_HWWIN_ID = 0;
FB_HWWIN_FORMAT = 6;
FB_HWLAYER_OUTPUTCOLOR = 0;
```

```
FB_MMAP_NAME = E_MMAP_ID_HW_CURSOR;
```

```
[FB_VIDEO]
```

```
FB_BUF_ID = 0
```

```
FB_BUF_CNT = 3
```

```
FB_HWWIN_FORMAT = 6
```

```
FB_WIDTH = 1920
```

```
FB_HEIGHT = 1080
```

```
FB_TIMMING_WIDTH = 1920
```

```
FB_TIMMING_HEIGHT = 1080
```

```
[FB_BUF_ID0_PATH]
```

```
FB_MOD = 34
```

```
FB_DEV = 1
```

```
FB_CHN = 0
```

```
FB_PORT = 0
```

```
FB_GWIN_ID = 7
```

注意：如果要添加多个 GUI 层（芯片支持的情况下），需要将新的 section name 加入到 m_fbdevList 中管理，否则不生效。比如当前 INI 配置有一个 FB_DEVICE 的 section 描述了一个 GUI 层的信息，可以再增加一个 FB_DEVICE1 的 section 来描述另外一个 GUI 层，然后把 FB_DEVICE1 这个 section name 加入到 m_fbdevList 即可，/dev/fbxxx 设备节点的增长顺序和 m_fbdevList 中的排序保持一致。

config.ini 中各配置参数说明如下：

参数	说明
FB_HWLAYER_ID	当前图形层的 ID（对应设备号）
FB_HWWIN_ID	硬件设备 ID（决定叠加到视频层设备 DISP 的哪一个输出 PATH 上）
FB_HWWIN_FORMAT	描述画布的像素格式
FB_HWLAYER_OUTPUTCOLOR	描述当前叠加层输出的颜色空间（YUV/RGB）
FB_WIDTH	描述可见区域范围（xres）
FB_HEIGHT	描述可见区域范围（yres）
FB_TIMING_WIDTH	描述显示设备分辨率（display width）
FB_TIMING_HEIGHT	描述显示设备分辨率（display height）
FB_MMAP_NAME	显存申请相关参数（默认为 E_MMAP_ID_FB）
FB_BUFFER_LEN	描述显存大小（单位 Kbyte）
FB_BUF_ID	用来关联 FB_BUF_ID0_PATH 字段
FB_BUF_CNT	用来计算叠加到视频处理单元的图形层的显存大小
FB_MOD	叠加图形层的视频处理单元 MODULE ID
FB_DEV	叠加图形层的视频处理单元 DEV ID
FB_CHN	叠加图形层的视频处理单元 CHN ID

参数	说明
FB_PORT	叠加图形层的视频处理单元 PORT ID

1.7.3 接口调用

接口调用分为两种，一种是基于标准 Linux Framebuff 的文件系统调用，一种是基于 MI_FB API 的接口调用。两种调用方式实现相同的功能。

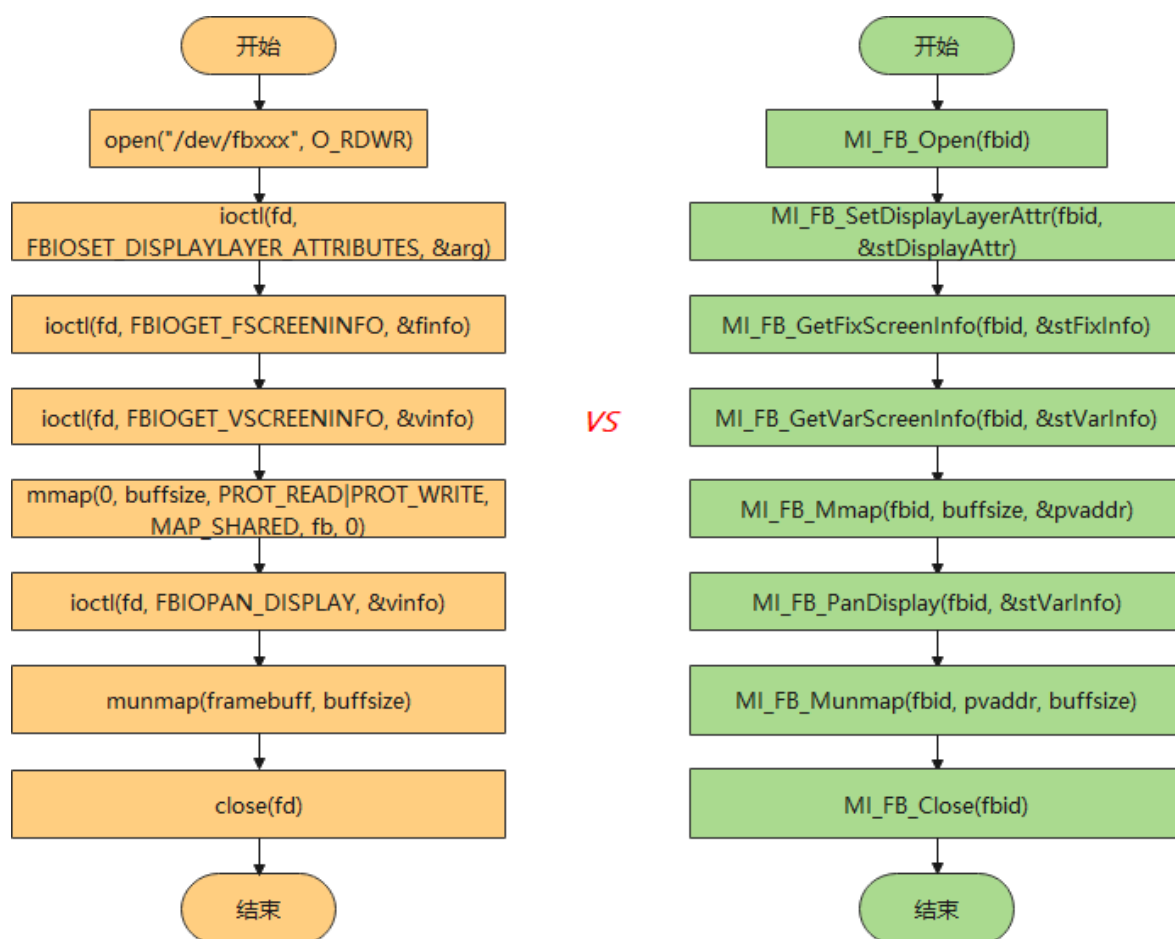


图 1-7 MI_FB 接口调用流程

MI_FB 的接口调用分为以下几步：

- 1、 打开指定的 FB 设备
- 2、 设置像素格式、可见区域等参数
- 3、 获取 MI_FB 分配到的显存信息
- 4、 将物理显存映射到虚拟内存空间
- 5、 调用 ioclt 或者 MI_FB API 设置 ColorKey、层间 Alpha、显示位置和区域等参数
- 6、 操作虚拟内存地址完成具体的绘制任务
- 7、 解除显存映射
- 8、 关闭 FB 设备
- 9、 退出应用

1.8.

实例介绍

本实例分别基于 Linux Framebuff 和 MI_FB API 实现在屏幕中间位置显示红、橙、蓝三个阶梯排列的色块，使用 ARGB8888 像素格式。

```
#include <stdio.h>
#include <fcntl.h>
#include <sys/ioctl.h>
#include <sys/mman.h>
#include <linux/fb.h>
#include "mi_sys.h"
#include "mi_disp.h"
#include "mi_disp_datatype.h"
#include "mi_fb.h"
#include "mi_fb_datatype.h"

static void *framebuffer = NULL;
#define LINUX_FRAMEBUFFER

void set_disp_dev(void)
{
    MI_DISP_PubAttr_t stPubAttr;
    memset(&stPubAttr, 0, sizeof(MI_DISP_PubAttr_t));
    stPubAttr.u32BgColor = YUYV_BLACK;
    stPubAttr.eIntfType == E_MI_DISP_INTF_TTL;
    MI_DISP_SetPubAttr(0, &stPubAttr);
    MI_PANEL_Init(E_MI_DISP_INTF_TTL);
}

#ifdef LINUX_FRAMEBUFFER
//基于Linux Framebuff实现
void drawRect_rgb32 (int x0, int y0, int width, int height, int color, struct fb_var_screeninfo *vinfo, struct fb_fix_screeninfo *finfo, char *fb)
{
    const int bytesPerPixel = 4;
    const int stride = finfo->line_length / bytesPerPixel;

    int *dest = (int *) (fb)
        + (y0 + vinfo->yoffset) * stride + (x0 + vinfo->xoffset);

    int x, y;
    for (y = 0; y < height; ++y)
    {
```

```
    for (x = 0; x < width; ++x)
    {
        dest[x] = color;
    }
    dest += stride;
}
}

int main(int argc, char **argv)
{
    const char *devfile = "/dev/fb0";
    int fbFd = 0;
    long int screensize = 0;

    MI_SYS_Init(0);

    //set disp pub
    set_disp_dev();
    fbFd = open(devfile, O_RDWR);
    if(fbFd == -1)
    {
        perror ("Error: cannot open framebuffer device");
        exit(1);
    }
    //get fb_fix_screeninfo
    if(ioctl(fbFd, FBIOGET_FSCREENINFO, &finfo) == -1)
    {
        perror("Error reading fixed information");
        exit(2);
    }
    //get fb_var_screeninfo
    if(ioctl(fbFd, FBIOGET_VSCREENINFO, &vinfo) == -1)
    {
        perror("Error reading variable information");
        exit(3);
    }
    screensize = finfo.smem_len;

    /* Map the device to memory */
    framebuffer = (char *)mmap(0, screensize, PROT_READ | PROT_WRITE, MAP_SHARED,
        fbFd, 0);
    if(framebuffer == MAP_FAILED)
    {
        perror ("Error: Failed to map framebuffer device to memory");
    }
}
```

```

    exit (4);
}
//draw red rectangle
drawRect_rgb32 (vinfo.xres / 8, vinfo.yres / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xffff0000, &vinfo, &finfo, framebuffer);

//draw DarkGoldenrod rectangle
drawRect_rgb32 (vinfo.xres * 3 / 8, vinfo.yres * 5 / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xffb8860b, &vinfo, &finfo, framebuffer);

//draw blue rectanble
drawRect_rgb32 (vinfo.xres * 5 / 8, vinfo.yres * 9 / 16,
    vinfo.xres / 4, vinfo.yres / 4, 0xff0000ff, &vinfo, &finfo, framebuffer);

munmap(framebuffer, screensize);
framebuffer = NULL;

close(fbFd);

return 0;
}
#else
//基于MI_FB_API实现
void drawRect_rgb32(int x0, int y0, int width, int height, int color, MI_FB_VarScreenInfo_t *vinfo, MI_FB_FixScreenInfo_t
*finfo, char *fb)
{
    const int bytesPerPixel = 4;
    const int stride = finfo->u32LineLength/ bytesPerPixel;

    int *dest = (int *) (fb)
        + (y0 + vinfo->u32Yoffset) * stride + (x0 + vinfo->u32Xoffset);

    int x, y;
    for (y = 0; y < height; ++y)
    {
        for (x = 0; x < width; ++x)
        {
            dest[x] = color;
        }
        dest += stride;
    }
}

int main(int argc, char **argv)

```

```
{
    MI_S32 s32Ret = 0;
    MI_U8 u8FbId = 0;
    MI_U32 screensize = 0;

    MI_SYS_Init(0);

    //set disp pub
    set_disp_dev();

    s32Ret = MI_FB_Open(u8FbId);
    if(s32Ret != MI_SUCCESS)
    {
        printf("open fb id:%d failed\n", u8FbId);
        exit(1);
    }
    s32Ret = MI_FB_GetFixScreenInfo(u8FbId, &stFixScreenInfo);
    if(s32Ret != MI_SUCCESS)
    {
        printf("get fix screen info failed\n");
        exit(2);
    }

    s32Ret = MI_FB_GetVarScreenInfo(u8FbId, &stVarScreenInfo);
    if(s32Ret != MI_SUCCESS)
    {
        printf("get var screen info failed\n");
        exit(3);
    }
    screensize = stFixScreenInfo.u32SmemLen;
    MI_FB_Mmap(u8FbId, screensize, &framebuffer);
    if(!framebuffer)
    {
        printf("mmap framebuffer failed\n");
        exit(4);
    }
    //draw red rectangle
    drawRect_rgb32 (stVarScreenInfo.u32Xres / 8, stVarScreenInfo.u32Yres/ 8,
        stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xffff0000, &stVarScreenInfo, &stFixScreenInfo,
framebuffer);

    //draw DarkGoldenrod rectangle
    drawRect_rgb32 (stVarScreenInfo.u32Xres * 3 / 8, stVarScreenInfo.u32Yres * 3 / 8,
        stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xffb8860b, &stVarScreenInfo, &stFixScreenInfo,
```

```
framebuffer);

    //draw blue rectanble
    drawRect_rgb32 (stVarScreenInfo.u32Xres * 5 / 8, stVarScreenInfo.u32Yres * 5 / 8,
        stVarScreenInfo.u32Xres / 4, stVarScreenInfo.u32Yres / 4, 0xff0000ff, &stVarScreenInfo, &stFixScreenInfo,
framebuffer);

    MI_FB_Munmap(u8FbId, framebuffer, screensize);
    framebuffer = NULL;

    MI_FB_Close(u8FbId);
    return 0;
}
#endif
```

2. API 参考

MI_FB 的用户态接口有两种体现形式:

1、 基于 Linux Framebuff 的文件操作，以 ioctl 的形式体现。

int ioctl(int fd, unsigned long cmd, CMD_DATA_TYPE *cmddata);

其中 CMD_DATA_TYPE 随参数 cmd 的变化而变化，MI_FB 支持以下 cmd。

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符，是调用 open 函数的返回值。	输入
cmd	主要的 cmd（命令控制字）如下： • FBIOGET_VSCREENINFO: 获取屏幕可变信息 • FBIOPUT_VSCREENINFO: 设置屏幕可变信息 • FBIOGET_FSCREENINFO: 获取屏幕固定信息 • FBIOPAN_DISPLAY: 设置 PAN 显示 • FBIOGET_CMAP: 获取颜色表信息 • FBIOPUT_CMAP: 设置颜色表信息 • FBIOGET_SCREEN_LOCATION: 获取叠加层显示区域 • FBIOSET_SCREEN_LOCATION: 设置叠加层显示区域 • FBIOGET_SHOW: 获取叠加层显示状态 • FBIOSET_SHOW: 设置叠加层显示状态 • FBIOGET_GLOBAL_ALPHA: 获取叠加层 Alpha 属性 • FBIOSET_GLOBAL_ALPHA: 设置叠加层 Alpha 属性 • FBIOGET_COLORKEY: 获取叠加层的 Colorkey 属性 • FBIOSET_COLORKEY: 设置叠加层的 Colorkey 属性 • FBIOGET_DISPLAYLAYER_ATTRIBUTES: 获取叠加层属性 • FBIOSET_DISPLAYLAYER_ATTRIBUTES: 设置叠加层属性 • FBIOGET_CURSOR_ATTRIBUTE: 获取鼠标层属性 • FBIOSET_CURSOR_ATTRIBUTE: 设置鼠标层属性	输入
cmddata	各 cmd 对应的数据类型： • 获取或设置屏幕可变信息: struct fb_var_screeninfo *类型 • 获取屏幕固定信息: struct fb_fix_screeninfo *类型 • 设置 PAN 显示: struct fb_var_screeninfo *类型 • 获取或设置屏幕叠加层坐标原点: MI_FB_Rectangle_t *类型 • 获取或设置叠加层显示状态: MI_BOOL *类型 • 获取或设置叠加层 Alpha: MI_FB_GlobalAlpha_t *类型 • 获取或设置叠加层 Colorkey: MI_FB_ColorKey_t *类型 • 获取或设置叠加层属性: MI_FB_DisplayLayerAttr_t *类型 • 获取或设置鼠标图层属性: MI_FB_CursorAttr_t *类型	输入/输出

2、 基于 MI_FB_API 以动态库或静态库的形式体现，目前支持以下 API。

API 名	功能
MI_FB_Open	打开 FB 设备
MI_FB_Close	关闭 FB 设备
MI_FB_Mmap	映射物理显存到虚拟内存空间
MI_FB_Munmap	取消映射物理显存到虚拟内存空间
MI_FB_GetVarScreenInfo	获取屏幕可变信息
MI_FB_SetVarScreenInfo	设置屏幕可变信息
MI_FB_GetFixScreenInfo	获取 FB 固定信息
MI_FB_PanDisplay	设置从虚拟分辨率中的不同偏移处开始显示
MI_FB_GetScreenLocation	获取叠加层在屏幕上的显示区域
MI_FB_SetScreenLocation	设置叠加层在屏幕上的显示区域
MI_FB_GetDisplayLayerAttr	获取图层信息
MI_FB_SetDisplayLayerAttr	设置图层信息
MI_FB_GetCursorAttr	获取鼠标图层信息
MI_FB_SetCursorAttr	设置鼠标图层信息
MI_FB_GetGlobalAlpha	获取叠加层的 Alpha 设置
MI_FB_SetGlobalAlpha	设置叠加层的 Alpha
MI_FB_GetColorKey	获取叠加层的 colorkey 设置
MI_FB_SetColorKey	设置叠加层的 colorkey
MI_FB_SetCmap	设置颜色表
MI_FB_GetCmap	获取颜色表信息
MI_FB_GetShow	获取叠加层显示状态
MI_FB_SetShow	显示或隐藏叠加层

在 1.4 章节已经介绍了两种体现形式的 API 在不同场景下的使用，用户可以根据实际情况选择使用其中一种 API 使用。

注意：两种调用方式不可以共存，用户只能二选其一。

以下内容将对两种体现形式的 API 接口进行详细描述。

2.1.

标准功能

2.1.1 FBIOWGET_VSCREENINFO

➤ 功能

获取屏幕的可变信息。

➤ 语法

```
int ioctl (int fd, FBIOWGET\_VSCREENINFO, struct fb_var_screeninfo *var);
```

➤ 描述

使用此接口获取屏幕的可变信息，主要包括分辨率和像素格式。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOWGET_VSCREENINFO	ioctl cmd	输入
var	可变信息结构体指针	输出

➤ 返回值

返回值 {
0 成功。
-1 失败。

➤ 依赖

头文件：fb.h

➤ 相关接口

[FBIOWPUT_VSCREENINFO](#)

2.1.2 FBIOWPUT_VSCREENINFO

➤ 功能

设置 Framebuffer 的屏幕分辨率和像素格式等。

➤ 语法

```
int ioctl (int fd, FBIOWPUT\_VSCREENINFO, struct fb_var_screeninfo *var);
```

➤ 描述

使用此接口设置屏幕分辨率、像素格式。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOPUT_VSCREENINFO	ioctl cmd	输入
var	可变信息结构体指针	输入

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件: fb.h

※ 注意

必须保证实际分辨率与偏移的和在虚拟分辨率范围内, 否则系统会自动调整实际分辨率的大小让其在虚拟分辨率范围内。

➤ 相关接口

[FBIOGET_VSCREENINFO](#)

2.1.3 FBIOGET_FSCREENINFO

➤ 功能

获取 Framebuffer 的固定信息。

➤ 语法

```
int ioctl (int fd, FBIOGET\_FSCREENINFO, struct fb\_fix\_screeninfo *fix);
```

➤ 描述

使用此接口获取 Framebuffer 固定信息, 包括显存起始物理地址、显存大小和行间距等。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_FSCREENINFO	ioctl cmd	输入
fix	固定信息结构体指针	输出

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：fb.h

※ 注意

无。

➤ 相关接口

无。

2.1.4 FBIOPAN_DISPLAY

➤ 功能

设置从虚拟分辨率中的不同偏移处开始显示。

➤ 语法

```
int ioctl (int fd, FBIOPAN_DISPLAY, struct fb_var_screeninfo *var);
```

➤ 描述

使用此接口设置从虚拟分辨率中的不同偏移处开始显示，实际的分辨率不变。
如图2.1所示（xres_virtual, yres_virtual）是虚拟分辨率，（xres, yres）是实际显示的分辨率，（xoffset, yoffset）是显示的偏移。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOPAN_DISPLAY	ioctl cmd	输入
var	可变信息结构体指针	输入

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：fb.h

※ 注意

必须保证实际分辨率与偏移的和在虚拟分辨率范围内，否则设置不成功。

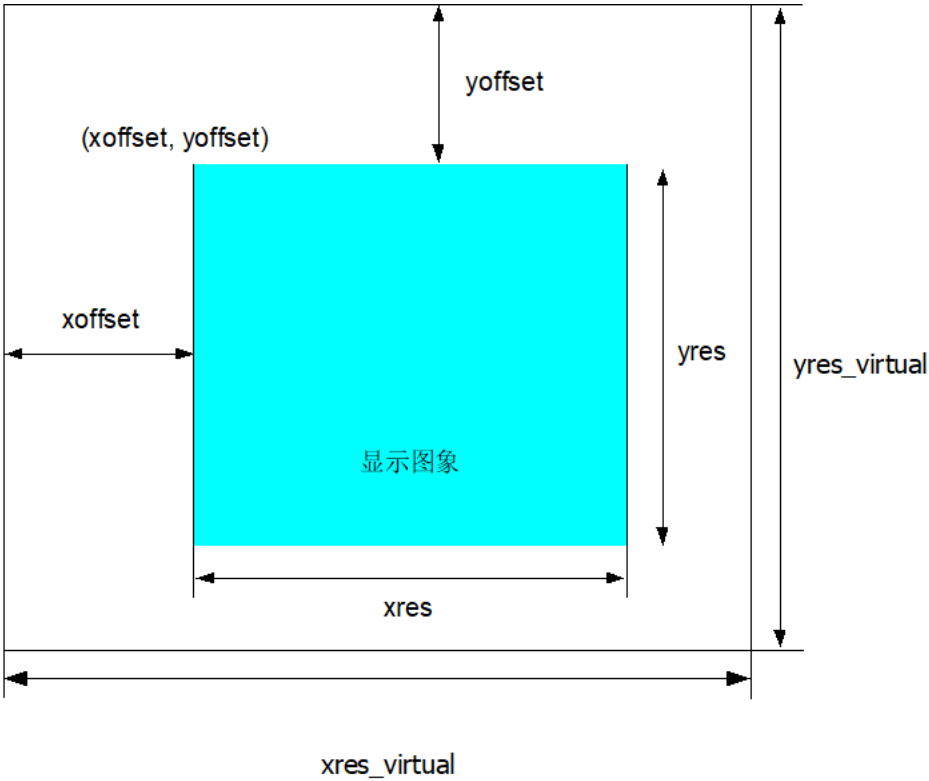


图 2-8 设置从虚拟分辨率中的不同偏移处开始显示

2.1.5 FBIO_WAITFORVSYNC

➤ 功能

等待屏幕垂直刷新完成(Vsync)信号。

➤ 语法

```
int ioctl (int fd, FBIO\_WAITFORVSYNC, u32 *zero);
```

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGETCMAP	ioctl cmd	输入
zero	确保传递的参数值为 0，否则它将无法工作	输入

➤ 返回值

返回值 { 0 成功。
-1 失败。

➤ 依赖

头文件: **fb.h**

2.1.6 FBIOGETCMAP

➤ 功能

获取颜色表信息。

➤ 语法

int ioctl (int fd, [FBIOGETCMAP](#), struct fb_cmap_user *cmap);

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGETCMAP	ioctl cmd	输入
cmap	描述颜色表信息	输出

➤ 返回值

返回值 { 0 成功。
-1 失败。

➤ 依赖

头文件: **fb.h**

➤ 举例

NA

➤ 相关主题

[FBIOPUTCMAP](#)
[MI_FB_SetCmap](#)

2.1.7 FBIOPUTCMAP

➤ 功能

设置颜色表信息。

➤ 语法

```
int ioctl (int fd, FBIOPUTCMAP, struct fb_cmap_user *cmap);
```

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOPUTCMAP	ioctl cmd	输入
cmap	描述颜色表信息	输入

➤ 返回值

返回值 {
0 成功。
-1 失败。

➤ 依赖

头文件: fb.h

➤ 举例

NA

➤ 相关主题

[FBIOGETCMAP](#)
[MI_FB_GetCmap](#)

2.2.

扩展功能

2.2.1 FBIOGET_SCREEN_LOCATION

➤ 功能

获取叠加层在屏幕上的显示区域

➤ 语法

```
int ioctl (int fd, FBIOGET\_SCREEN\_LOCATION, MI_FB_Rectangle_t* pstRectangle);
```

➤ 描述

使用此接口获取叠加层在屏幕上的显示区域

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_SCREEN_LOCATION	ioctl cmd	输入
pstRectangle	MI_FB_Rectangle_t结构体指针, 包含(x,y)坐标和	输出

	width,height。表示叠加层在屏幕上的显示区域	
--	-----------------------------	--

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件: mi_fb_datatype.h

➤ 相关接口

[FBIOSET_SCREEN_LOCATION](#)

2.2.2 FBIOSET_SCREEN_LOCATION

➤ 功能

设置叠加层在屏幕上的显示区域

➤ 语法

int ioctl (int fd, [FBIOSET_SCREEN_LOCATION](#), MI_FB_Rectangle_t* pstRectangle);

➤ 描述

使用此接口设置叠加层在屏幕上的显示区域

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_SCREEN_LOCATION	ioctl cmd	输入
pstRectangle	MI_FB_Rectangle_t结构体指针, 包含(x,y)坐标和width,height。表示叠加层在屏幕上的显示区域	输入

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 举例

无。

※ 注意

无

➤ 相关接口

[FBIOGET_SCREEN_LOCATION](#)

2.2.3 FBIOGET_SHOW

➤ 功能

获取当前叠加层的显示状态。

➤ 语法

```
int ioctl (int fd, FBIOGET\_SHOW, MI_BOOL *bShow);
```

➤ 描述

使用此接口获取当前叠加层显示状态。

➤ 形参

参数名称	描述	输入/输出
Fd	Framebuffer 设备文件描述符	输入
FBIOGET_SHOW	ioctl cmd	输入
bShow	指示当前叠加层的状态： • *bShow = MS_TRUE: 当前叠加层处于显示状态 • *bShow = MS_FALSE: 当前叠加层处于隐藏状态	输出

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 相关接口

[FBIOSET_SHOW](#)

2.2.4 FBIOSET_SHOW

➤ 功能

显示或隐藏该叠加层。

➤ 语法

```
int ioctl (int fd, FBIOSET\_SHOW, MI_BOOL *bShow);
```

➤ 描述

使用此接口设置叠加层显示状态：显示或隐藏。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_SHOW	ioctl cmd	输入
bShow	该叠加层的显示状态： • *bShow = MS_TRUE: 显示当前叠加层 • *bShow = MS_FALSE: 隐藏当前叠加层	输入

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 相关接口

[FBIOGET_SHOW](#)

2.2.5 FBIOGET_GLOBAL_ALPHA

➤ 功能

获取叠加层 Alpha 设置。

➤ 语法

```
int ioctl (int fd, FBIOGET\_GLOBAL\_ALPHA, MI\_FB\_GlobalAlpha\_t *pstAlpha);
```

➤ 描述

使用此接口获取当前叠加层的 Alpha 设置。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_GLOBAL_ALPHA	ioctl cmd	输入
pstAlpha	MI_FB_GlobalAlpha_t 结构体指针	输出

➤ 返回值

返回值 {
· 0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 相关接口

[FBIOSET_GLOBAL_ALPHA](#)

2.2.6 FBIOSET_GLOBAL_ALPHA

➤ 功能

设置叠加层的 Alpha。

➤ 语法

int ioctl (int fd, [FBIOSET_GLOBAL_ALPHA](#), [MI_FB_GlobalAlpha_t](#) *pstAlpha);

➤ 描述

使用此接口设置当前叠加层的 Alpha 功能。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_GLOBAL_ALPHA	ioctl cmd	输入
pstAlpha	MI_FB_GlobalAlpha_t 结构体指针	输入

➤ 返回值

返回值 {
· 0 成功。

-1 失败

➤ 依赖

头文件: `mi_fb_datatype.h`

➤ 相关接口

[FBIOGET_GLOBAL_ALPHA](#)

2.2.7 FBIOGET_COLORKEY

➤ 功能

获取叠加层的 `colorkey`。

➤ 语法

```
int ioctl (int fd, FBIOGET\_COLORKEY, MI\_FB\_ColorKey\_t* pstColorKey);
```

➤ 描述

使用此接口获取叠加层的 `colorkey`。

➤ 形参

参数名称	描述	输入/输出
Fd	Framebuffer 设备文件描述符	输入
FBIOGET_COLORKEY	ioctl cmd	输入
pstColorKey	MI_FB_ColorKey_t 结构体指针	输出

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件: `mi_fb_datatype.h`

➤ 相关接口

[FBIOSET_COLORKEY](#)

2.2.8 FBIOSET_COLORKEY

➤ 功能

设置叠加层的 `colorkey`。

➤ 语法

```
int ioctl (int fd, FBIOSET\_COLORKEY, MI\_FB\_ColorKey\_t* pstColorKey);
```

➤ 描述

使用此接口设置当前叠加层的 colorkey 功能。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_COLORKEY	ioctl cmd	输入
pstColorKey	MI_FB_ColorKey_t 结构体指针	输入

➤ 返回值

返回值 { 0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 相关接口

[FBIOGET_COLORKEY](#)

2.2.9 FBIOSET_DISPLAYLAYER_ATTRIBUTES

➤ 功能

设置图层信息。

➤ 语法

```
int ioctl (int fd, FBIOSET\_DISPLAYLAYER\_ATTRIBUTES,  
MI\_FB\_DisplayLayerAttr\_t* pstLayerInfo);
```

➤ 描述

此接口用于设置图层信息，包括图层在屏幕的显示区域、画布分辨率、显存分辨率、屏幕显示分辨率以及是否使能预乘。以上信息的更详细说明见 MI_FB_DisplayLayerAttr_t 的描述。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSET_DISPLAYLAYER_ATTRIBUTES	ioctl cmd	输入

pstLayerInfo	MI_FB_DisplayLayerAttr_t结构体 指针	输出
--------------	-----------------------------------	----

➤ 返回值

返回值 { 0 成功。
-1 失败

➤ 依赖

头文件: mi_fb_datatype.h

➤ 相关接口

[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#)

2.2.10 FBIOGET_DISPLAYLAYER_ATTRIBUTES

➤ 功能

获取图层信息。

➤ 语法

```
int ioctl (int fd, FBIOGET\_DISPLAYLAYER\_ATTRIBUTES,  
           MI\_FB\_DisplayLayerAttr\_t * pstLayerInfo);
```

➤ 描述

用于获取图层信息，包括图层在屏幕的显示区域、显存分辨率、屏幕显示分辨率以及是否使能预乘。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_DISPLAYLAYER_ATTRIBUTES	ioctl cmd	输入
pstLayerInfo	图层信息结构体指针	输出

➤ 返回值

返回值 { 0 成功。
-1 失败

➤ 依赖

头文件: mi_fb_datatype.h

➤ 相关接口

[FBIOGET_DISPLAYLAYER_ATTRIBUTES](#)

2.2.11 FBIOGET_CURSOR_ATTRIBUTE

➤ 功能

获取鼠标图层信息。

➤ 语法

```
int ioctl (int fd, FBIOGET\_CURSOR\_ATTRIBUTE,  
           MI\_FB\_CursorAttr\_t *pstCursorAttr)
```

➤ 描述

获取鼠标图层信息，包括hotspot、位置、Alpha、Colorkey、是否可见。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOGET_CURSOR_ATTRIBUTE	ioctl cmd	输入
pstCursorAttr	MI_FB_CursorAttr_t结构体指针	输出

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件：mi_fb_datatype.h

➤ 相关接口

[FBIOSET_CURSOR_ATTRIBUTE](#)

2.2.12 FBIOSET_CURSOR_ATTRIBUTE

➤ 功能

设置鼠标图层信息。

➤ 语法

```
int ioctl (int fd, FBIOSET\_CURSOR\_ATTRIBUTE,  
           MI\_FB\_CursorAttr\_t *pstCursorAttr)
```

➤ 描述

设置鼠标图层信息, 包括hotspot、位置、Alpha、Colorkey、是否可见。

➤ 形参

参数名称	描述	输入/输出
fd	Framebuffer 设备文件描述符	输入
FBIOSSET_CURSOR_ATTRIBUTE	ioctl cmd	输入
pstCursorAttr	MI_FB_CursorAttr_t结构体指针	输出

➤ 返回值

返回值 {
0 成功。
-1 失败

➤ 依赖

头文件: mi_fb_datatype.h

➤ 相关接口

[FBIOSGET_CURSOR_ATTRIBUTE](#)

2.3.

MI_FB_API 参考

2.3.1 MI_FB_Open

➤ 功能

该API功能和open framebuffer设备节点相同

➤ 语法

MI_S32 [MI_FB_Open](#) (MI_U32 u32FbId);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入

➤ 返回值

返回值 {
0 成功。
非0 失败, 详情参照[错误码](#)。

➤ 依赖

头文件: `mi_fb.h`

库文件: `libmi_fb.a`

➤ 举例

```
MI_S32 s32Ret = MI_SUCCESS;
MI_U32 u32FbId = 0;
void *framebuff = NULL;
MI_U32 u32ScreenSize;
MI_FB_VarScreenInfo_t stVarScreenInfo;
MI_FB_FixScreenInfo_t stFixScreenInfo;

memset(&stVarScreenInfo, 0, sizeof(MI_FB_VarScreenInfo_t));
memset(&stFixScreenInfo, 0, sizeof(MI_FB_FixScreenInfo_t));

s32Ret = MI_FB_Open(u32FbId);
if(MI_SUCCESS == s32Ret)
{
    MI_FB_GetFixScreenInfo(u32FbId, &stFixScreenInfo);
    MI_FB_GetVarScreenInfo(u32FbId, &stVarScreenInfo);
    u32ScreenSize = stFixScreenInfo.u32SmemLen;
    if(u32ScreenSize)
        MI_FB_Mmap(u32FbId, u32ScreenSize, &framebuff);
    if(framebuff)
    {
        draw(x, y, framebuff);
        stVarScreenInfo.u32Xoffset = 0;
        stVarScreenInfo.u32Yoffset = stVarScreenInfo.u32Yres;
        MI_FB_PanDisplay(u32FbId, &stVarScreenInfo);
    }
    MI_FB_Close(u32FbId);
}
```

➤ 相关主题

[MI_FB_Close](#)

2.3.2 MI_FB_Close

➤ 功能

该API功能和close framebuff设备节点相同

➤ 语法

MI_S32 [MI_FB_Close](#) (MI_U32 u32FbId);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入

➤ 返回值

返回值 {
· 0 成功。
· 非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_Open](#)

2.3.3 MI_FB_Mmap

➤ 功能

该API功能和framebuff mmap相同

➤ 语法

MI_S32 [MI_FB_Mmap](#) (MI_U32 u32FbId, MI_U32 u32size, void **pVirAddr);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
u32size	buff 大小	输入
pVirAddr	返回映射后的虚拟地址	输出

➤ 返回值

返回值 {
· 0 成功。
· 非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: `mi_fb.h`
库文件: `libmi_fb.a`

➤ 举例

NA

➤ 相关主题

NA

2.3.4 MI_FB_Munmap

➤ 功能

该API功能和framebuff munmap相同。

➤ 语法

MI_S32 [MI_FB_Munmap](#) (MI_U32 u32FbId, void *pVirAddr, MI_U32 u32Size);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pVirAddr	映射过的虚拟地址	输入
u32Size	buff 大小	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

NA

2.3.5 MI_FB_GetVarScreenInfo

➤ 功能

该 API 功能和 [FBIOGET_VSCREENINFO](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetVarScreenInfo](#) (MI_U32 u32FbId, [MI_FB_VarScreenInfo_t](#) *pstVarScreenInfo);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstVarScreenInfo	描述屏幕可变信息	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetVarScreenInfo](#)
[FBIOGET_VSCREENINFO](#)

2.3.6 MI_FB_SetVarScreenInfo

➤ 功能

该 API 功能和 [FBIOPUT_VSCREENINFO](#) 相同

➤ 语法

MI_S32 [MI_FB_SetVarScreenInfo](#) (MI_U32 u32FbId, [MI_FB_VarScreenInfo_t](#) *pstVarScreenInfo);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstVarScreenInfo	描述屏幕可变信息	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetVarScreenInfo](#)
[FBIOPUT_VSCREENINFO](#)

2.3.7 MI_FB_GetFixScreenInfo

➤ 功能

该 API 功能和 [FBIOPUT_FSCREENINFO](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetFixScreenInfo](#) (MI_U32 u32FbId, [MI_FB_FixScreenInfo_t](#) *pstFixScreenInfo);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstFixScreenInfo	描述屏幕固定信息	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[FBIOPUT_FSCREENINFO](#)

2.3.8 MI_FB_PanDisplay

➤ 功能

该 API 功能和 [FBIOPAN_DISPLAY](#) 相同。

➤ 语法

MI_S32 [MI_FB_PanDisplay](#) (MI_U32 u32FbId, [MI_FB_VarScreenInfo_t](#) *pstVarScreenInfo);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstVarScreenInfo	描述屏幕可变信息	输入

➤ 返回值

返回值 {
· 0 成功。
· 非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: [mi_fb.h](#) [mi_fb_datatype.h](#)
库文件: [libmi_fb.a](#)

※ 注意

NA

➤ 举例

NA

➤ 相关主题

[FBIOPAN_DISPLAY](#)

2.3.9 MI_FB_WaitForVsync

➤ 功能

该 API 功能和 [FBIO_WAITFORVSYNC](#) 相同。

➤ 语法

MI_S32 [MI_FB_WaitForVsync](#) (MI_U32 u32FbId);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: `mi_fb.h` `mi_fb_datatype.h`
库文件: `libmi_fb.a`

➤ 举例

NA

2.3.10 MI_FB_GetScreenLocation

➤ 功能

该 API 功能和 [FBIOGET_SCREEN_LOCATION](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetScreenLocation](#) (MI_U32 u32FbId, [MI_FB_Rectangle_t](#) *pstWinRect);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstWinRect	描述叠加层在屏幕上的显示区域	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: `mi_fb.h` `mi_fb_datatype.h`
库文件: `libmi_fb.a`

➤ 举例

NA

➤ 相关主题

[MI_FB_SetScreenLocation](#)
[FBIOGET_SCREEN_LOCATION](#)

2.3.11 MI_FB_SetScreenLocation

➤ 功能

该 API 功能和 [FBIOSET_SCREEN_LOCATION](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetScreenLocation](#) (MI_U32 u32FbId, [MI_FB_Rectangle_t](#) *pstWinRect);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstWinRect	描述叠加层在屏幕上的显示区域	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h

库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetScreenLocation](#)

[FBIOSET_SCREEN_LOCATION](#)

2.3.12 MI_FB_GetDisplayLayerAttr

➤ 功能

该 API 功能和 [FBIOGET_DISPLAYLAYER_ATTRIBUTES](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetDisplayLayerAttr](#) (MI_U32 u32FbId, MI_FB_DisplayLayerAttr_t *pstDisplayLayerAttr);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入

pstDisplayLayerAttr	描述图层信息	输出
---------------------	--------	----

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetDisplayLayerAttr](#)
[FBIOGET_DISPLAYLAYER_ATTRIBUTES](#)

2.3.13 MI_FB_SetDisplayLayerAttr

➤ 功能

该 API 功能和 [FBIOSET_DISPLAYLAYER_ATTRIBUTES](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetDisplayLayerAttr](#) (MI_U32 u32FbId, [MI_FB_DisplayLayerAttr_t](#) *pstDisplayLayerAttr);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstDisplayLayerAttr	描述图层信息	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h

库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetDisplayLayerAttr](#)
[FBIOSET_DISPLAYLAYER_ATTRIBUTES](#)

2.3.14 MI_FB_GetCursorAttr

➤ 功能

该 API 功能和 [FBIOGET_CURSOR_ATTRIBUTE](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetCursorAttr](#) (MI_U32 u32FbId, [MI_FB_CursorAttr_t](#) *pstCursorAttr);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstCursorAttr	描述鼠标图层信息	输出

➤ 返回值

返回值 {
0 成功。
非 0 失败, 详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetCursorAttr](#)
[FBIOGET_CURSOR_ATTRIBUTE](#)

2.3.15 MI_FB_SetCursorAttr

➤ 功能

该 API 功能和 [FBIOSET_CURSOR_ATTRIBUTE](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetCursorAttr](#) (MI_U32 u32FbId, [MI_FB_CursorAttr_t](#) *pstCursorAttr);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstCursorAttr	描述鼠标图层信息	输入

➤ 返回值

返回值 {
· 0 成功。
· 非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetCursorAttr](#)
[FBIOSET_CURSOR_ATTRIBUTE](#)

2.3.16 MI_FB_GetGlobalAlpha

➤ 功能

该 API 功能和 [FBIOGET_GLOBAL_ALPHA](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetGlobalAlpha](#) (MI_U32 u32FbId, [MI_FB_GlobalAlpha_t](#) *pstAlpha);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstAlpha	描述叠加层的 Alpha	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetGlobalAlpha](#)
[FBIOGET_GLOBAL_ALPHA](#)

2.3.17 MI_FB_SetGlobalAlpha

➤ 功能

该 API 功能和 [FBIOSET_GLOBAL_ALPHA](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetGlobalAlpha](#) (MI_U32 u32FbId, [MI_FB_GlobalAlpha_t](#) *pstAlpha);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstAlpha	描述叠加层的 Alpha	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h mi_fb_datatype.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetGlobalAlpha](#)
[FBIOSET_GLOBAL_ALPHA](#)

2.3.18 MI_FB_GetColorKey

➤ 功能

该 API 功能和 [FBIOGET_COLORKEY](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetColorKey](#) (MI_U32 u32FbId, [MI_FB_ColorKey_t](#) *pstClrKey);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstClrKey	描述叠加层的 colorkey 信息	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetColorKey](#)
[FBIOGET_COLORKEY](#)

2.3.19 MI_FB_SetColorKey

➤ 功能

该 API 功能和 [FBIOSET_COLORKEY](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetColorKey](#) (MI_U32 u32FbId, [MI_FB_ColorKey_t](#) *pstClrKey);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstClrKey	描述叠加层的 colorkey 信息	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetColorKey](#)
[FBIOSET_COLORKEY](#)

2.3.20 MI_FB_SetCmap

➤ 功能

该 API 功能和 [FBIOPUTCMAP](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetCmap](#) (MI_U32 u32FbId, [MI_FB_Cmap_t](#) *pstCmap);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstCmap	描述颜色表信息	输入

➤ 返回值

返回值 {
0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: mi_fb.h mi_fb_datatype.h
库文件: libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetCmap](#)

2.3.21 MI_FB_GetCmap

➤ 功能

该 API 功能和 [FBIOGETCMAP](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetCmap](#) (MI_U32 u32FbId, [MI_FB_Cmap_t](#) *pstCmap);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pstCmap	描述颜色表信息	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件: [mi_fb.h](#) [mi_fb_datatype.h](#)
库文件: [libmi_fb.a](#)

➤ 举例

NA

➤ 相关主题

[MI_FB_SetCmap](#)

2.3.22 MI_FB_GetShow

➤ 功能

该 API 功能和 [FBIOGET_SHOW](#) 相同。

➤ 语法

MI_S32 [MI_FB_GetShow](#) (MI_U32 u32FbId, MI_BOOL *pbShown);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
pbShown	叠加层显示状态	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_SetShow](#)
[FBIOGET_SHOW](#)

2.3.23 MI_FB_SetShow

➤ 功能

该 API 功能和 [FBIOSET_SHOW](#) 相同。

➤ 语法

MI_S32 [MI_FB_SetShow](#) (MI_U32 u32FbId, MI_BOOL bShown);

➤ 形参

参数名称	描述	输入/输出
u32FbId	FB 设备号	输入
bShown	1: 显示 0: 隐藏	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，详情参照[错误码](#)。

➤ 依赖

头文件：mi_fb.h
库文件：libmi_fb.a

➤ 举例

NA

➤ 相关主题

[MI_FB_GetShow](#)
[FBIOSET_SHOW](#)

3. 数据类型

Linux Framebuff 标准中定义的数据类型:

数据类型	定义
struct fb_bitfield	位域信息,用于设置像素格式
struct fb_var_screeninfo MI_FB_VarScreenInfo_t	可变的屏幕信息
struct fb_fix_screeninfo	固定的屏幕信息
struct fb_cmap_user	描述颜色表信息

MI_FB 中定义的数据类型:

数据类型	定义
MI_FB_BitField_t	同标准定义
MI_FB_VarScreenInfo_t	同标准定义
MI_FB_FixScreenInfo_t	同标准定义
MI_FB_ColorFmt_e	FB 支持的像素格式
MI_FB_OuputColorSpace_e	叠加层输出的 color space
MI_FB_DstDisplayPlane_e	FB 叠加层输出到 display chain 的目的模组
MI_FB_DisplayLayerAttrMaskBit_e	指定 MI_FB_DisplayLayerAttr_t 中需要更新的属性
MI_FB_CursorAttrMaskBit_e	指定 MI_FB_CursorAttr_t 中需要更新的属性
MI_FB_DisplayLayerAttr_t	描述叠加层的属性信息
MI_FB_GlobalAlpha_t	Alpha 信息结构体
MI_FB_ColorKey_t	Colorkey 信息结构体
MI_FB_CursorImage_t	描述鼠标图标数据信息
MI_FB_CursorAttr_t	描述鼠标图层信息
MI_FB_Rectangle_t	描述叠加层的显示区域
MI_FB_Cmap_t	描述颜色表信息

注: 本节已涵盖各重要的数据类型, 部分未列出数据类型请参见 [mi_fb_datatype.h](#)

3.1.

标准定义

3.1.1 struct fb_bitfield

➤ 说明

位域信息，用于设置像素格式。

➤ 定义

```
struct fb_bitfield
{
    _u32 offset;          /* beginning of bitfield */
    _u32 length;          /* length of bitfield */
    _u32 msb_right;       /* != 0: Most significant bit is right */
};
```

➤ 成员

成员名称	描述	支持情况
offset	颜色分量起始比特位。	支持。
length	颜色分量所占比特长度。	支持。
msb_right	右边的比特是否为最高有效位。	只支持该位为 0，即最左边的 bit 为最高有效位。

➤ 相关数据类型及接口

无。

3.1.2 struct fb_var_screeninfo

➤ 说明

可变的屏幕信息。

➤ 定义

```
struct fb_var_screeninfo
{
    _u32 xres;              /* visible resolution */
    _u32 yres;
    _u32 xres_virtual;      /* virtual resolution */
    _u32 yres_virtual;
    _u32 xoffset;           /* offset from virtual to visible */
    _u32 yoffset;           /* resolution */

    _u32 bits_per_pixel;    /* guess what */
    _u32 grayscale;        /* != 0 Graylevels instead of colors */
    struct fb_bitfield red;  /* bitfield in fb mem if true color, */
    struct fb_bitfield green; /* else only length is significant */
    struct fb_bitfield blue;
```

```

struct fb_bitfield transp; /* transparency */

_u32 nonstd;                /* != 0 Non standard pixel format */

_u32 activate;              /* see FB_ACTIVATE_* */

_u32 height;                /* height of picture in mm */
_u32 width;                 /* width of picture in mm */

_u32 accel_flags;           /* (OBSOLETE) see fb_info.flags */

/* Timing: All values in pixclocks, except pixclock (of course) */
_u32 pixclock;              /* pixel clock in ps (pico seconds) */
_u32 left_margin;           /* time from sync to picture */
_u32 right_margin;          /* time from picture to sync */
_u32 upper_margin;          /* time from sync to picture */
_u32 lower_margin;
_u32 hsync_len;             /* length of horizontal sync */
_u32 vsync_len;             /* length of vertical sync */
_u32 sync;                  /* see FB_SYNC_* */
_u32 vmode;                 /* see FB_VMODE_* */
_u32 rotate;                /* angle we rotate counter clockwise */
_u32 reserved[5];           /* Reserved for future compatibility */
};

```

➤ 成员

成员名称	描述	支持情况
xres	可见屏幕宽度（像素数）。	支持。
yres	可见屏幕高度（像素数）。	支持。
xres_virtual	虚拟屏幕宽度（显存中图像宽度），当该值小于 xres 时会修改 xres，使 xres 值与该值相等。	支持。
yres_virtual	虚拟屏幕高度（显存中图像高度），当该值小于 yres 时会修改 yres，使 yres 值与该值相等。结合 xres_virtual，可以用来快速水平或垂直平移图像。	支持。
xoffset	在 x 方向上的偏移像素数。	支持，默认为 0。
yoffset	在 y 方向上的偏移像素数。	支持，默认为 0。
bits_per_pixel	每个像素所占的比特数。	支持。
grayscale	灰度级。	不支持，缺省值为 0，表示彩色。
red	颜色分量中红色的位域信息。	支持。

成员名称	描述	支持情况
green	颜色分量中绿色的位域信息。	支持。
blue	颜色分量中蓝色的位域信息。	支持。
transp	颜色分量中 alpha 分量的位域信息。	支持。
nonstd	是否为标准像素格式。	不支持，缺省值为 0，表示支持标准像素格式。
activate	设置生效的时刻。	不支持，缺省值为 FB_ACTIVATE_NOW，表示设置立刻生效。
height	屏幕高，单位为 mm。	不支持，缺省值为 - 1。
width	屏幕宽，单位为 mm。	不支持，缺省值为 - 1。
accel_flags	加速标志。	不支持，缺省值为 - 1。
pixclock	显示一个点需要的时间，单位为ns。	不支持，缺省值为 - 1。
left_margin	分别是左消隐信号、右消隐信号、水平同步时长，这三个值之和等于水平回扫时间，单位为点时钟。	不支持，缺省值为64。
right_margin		
hsync_len		
upper_margin	分别是上消隐信号、下消隐信号、垂直同步时长，这三个值之和等于垂直回扫时间，单位为点时钟。	不支持，缺省值为 - 1。
lower_margin		
vsync_len		
sync	同步信号方式。	不支持，缺省值为 - 1。
vmode	扫描模式。	不支持，缺省值为 - 1。
rotate	顺时针旋转的角度。	不支持，缺省值为 0，表示无旋转。

※ 注意事项

高清设备图形层默认的分辨率为 1280x720；标清设备图形层默认的分辨率为720x576，鼠标层默认的分辨率为 128x128。像素格式为 ARGB1555。

➤ 相关数据类型及接口

[struct fb_bitfield](#)
[FBIOGET_VSCREENINFO](#)
[FBIOPUT_VSCREENINFO](#)

3.1.3 struct fb_fix_screeninfo

➤ 说明

固定的屏幕信息。

➤ 定义

```
struct fb_fix_screeninfo
{
    char id[16]; /* identification string eg "TT Builtin" */ unsigned long
    smem_start; /* Start of frame buffer mem (physical
```

```

        address) */
    _u32 smem_len;          /* Length of frame buffer mem */
    _u32 type;              /* see FB_TYPE_* */
    _u32 type_aux;          /* Interleave for interleaved Planes */
    _u32 visual;            /* see FB_VISUAL_* */
    _u16 xpanstep;          /* zero if no hardware panning */
    _u16 ypanstep;          /* zero if no hardware panning */
    _u16 ywrapstep;         /* zero if no hardware ywrap */
    _u32 line_length;       /* length of a line in bytes */ unsigned long
    mmio_start;             /* Start of Memory Mapped I/O (physical
        address) */
    _u32 mmio_len;          /* Length of Memory Mapped I/O */
    _u32 accel; /* Indicate to driver which specific chip/card we have */
    _u16 reserved[3];       /* Reserved for future compatibility */
};

```

➤ 成员

成员名称	描述	支持情况
id	设备驱动名称。	支持。
smem_start	显存起始物理地址。	支持。
smem_len	显存大小。	支持。
type	显卡类型。	固定为 FB_TYPE_PACKED_PIXELS，表示像素值紧密排列。
type_aux	附加类型。	不支持，在 FB_TYPE_PACKED_PIXELS 显卡类型下无含义。
visual	色彩模式。	不支持，默认为 FB_VISUAL_TRUECOLOR，真彩色。
xpanstep	支持水平方向上的 PAN 显示： 0：不支持。 非 0：支持，此时该值用于表示在水平方向上每步进的像素值。	固定为 1。
ypanstep	支持垂直方向上的 PAN 显示： 0：不支持。 非 0：支持，此时该值用于表示在垂直方向上每步进的像素值。	固定为 1。
ywrapstep	该方式类似于 ypanstep，不同之处在于：当其显示到底部时，能回到显存的开始处进行显示。	不支持，默认为 0。

成员名称	描述	支持情况
line_length	每行字节数。	支持。
mmio_start	显存映射 I/O 首地址。	不支持，默认为 0。
mmio_len	显存映射 I/O 长度。	不支持，默认为 0。
accel	显示所支持的硬件加速设备。	不支持，默认为 FB_ACCEL_NONE，无加速设备。
reserved	保留。	不支持，缺省值为 0。

※ 注意事项

无。

➤ 相关数据类型及接口

[FBIOGET_FSCREENINFO](#)

3.1.4 struct fb_cmap_user

➤ 说明

描述颜色表信息。

➤ 定义

```
struct fb_cmap_user
{
    __u32 start;           /* First entry */
    __u32 len;             /* Number of entries */
    __u32 __user *red;     /* Red values */
    __u32 __user *green;
    __u32 __user *blue;
    __u32 __user *transp; /* transparency, can be NULL*/
};
```

➤ 成员

成员名称	描述	支持情况
start	第一个元素的入口	支持
len	元素个数	支持
red	红色分量起始位置	支持
green	绿色分量起始位置	支持
blue	蓝色分量起始位置	支持
transp	透明度分量起始位置	支持

➤ 相关数据类型及接口

无。

3.2.

MI_FB 数据类型定义

3.2.1 MI_FB_BitField_t

➤ 说明

描述信息和标准定义相同，参考struct fb_bitfield

➤ 相关数据类型及接口

[MI_FB_GetVarScreenInfo](#)

[MI_FB_SetVarScreenInfo](#)

3.2.2 MI_FB_VarScreenInfo_t

➤ 说明

描述信息和标准定义相同，参考struct fb_var_screeninfo

➤ 相关数据类型及接口

[MI_FB_GetVarScreenInfo](#)

[MI_FB_SetVarScreenInfo](#)

3.2.3 MI_FB_FixScreenInfo_t

➤ 说明

描述信息和标准定义相同，参考struct fb_fix_screeninfo

➤ 相关数据类型及接口

[MI_FB_GetFixScreenInfo](#)

3.2.4 MI_FB_ColorFmt_e

➤ 说明

FB支持的像素格式集合

➤ 定义

```
typedef enum
{
    E_MI_FB_COLOR_FMT_RGB565      = 1,
    E_MI_FB_COLOR_FMT_ARGB4444    = 2,
    E_MI_FB_COLOR_FMT_I8          = 4,
    E_MI_FB_COLOR_FMT_ARGB8888    = 5,
    E_MI_FB_COLOR_FMT_ARGB1555    = 6,
    E_MI_FB_COLOR_FMT_YUV422      = 9,
    E_MI_FB_COLOR_FMT_I4          = 13,
```

```

        E_MI_FB_COLOR_FMT_I2          = 14,
        /// Invalid color format.
        E_MI_FB_COLOR_FMT_INVALID
    }MI_FB_ColorFmt_e;

```

➤ 成员

成员名称	描述
E_MI_FB_COLOR_FMT_RGB565	RGB565 格式。
E_MI_FB_COLOR_FMT_ARGB4444	ARGB4444 格式。
E_MI_FB_COLOR_FMT_I8	I8
E_MI_FB_COLOR_FMT_ARGB8888	ARGB8888
E_MI_FB_COLOR_FMT_ARGB1555	ARGB1555
E_MI_FB_COLOR_FMT_YUV422	YUV422
E_MI_FB_COLOR_FMT_I4	I4
E_MI_FB_COLOR_FMT_I2	I2
E_MI_FB_COLOR_FMT_INVALID	无效像素格式

➤ 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#)

3.2.5 MI_FB_OuputColorSpace_e

➤ 说明

FB叠加层输出的color space

➤ 定义

```

typedef enum
{
    E_MI_FB_OUTPUT_RGB = 0,
    E_MI_FB_OUTPUT_YUV = 1
}MI_FB_OutputClrSpace_e;

```

➤ 成员

成员名称	描述
E_MI_FB_OUTPUT_RGB	RGB ColorSpace
E_MI_FB_OUTPUT_YUV	YUV ColorSpace

➤ 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#)

3.2.6 MI_FB_DstDisplayPlane_e

➤ 说明

FB叠加层输出到display chain的目的模组，通过配置不同的display chain，可以选择叠加层参与的图像调整过程。

➤ 定义

```
typedef enum
{
    E_MI_FB_DST_IP0 = 0,
    E_MI_FB_DST_IP0_SUB = 1,
    E_MI_FB_DST_MIXER2VE = 2,
    E_MI_FB_DST_OP0 = 3,
    E_MI_FB_DST_VOP = 4,
    E_MI_FB_DST_IP1 = 5,
    E_MI_FB_DST_IP1_SUB = 6,
    E_MI_FB_DST_MIXER2OP = 7,
    E_MI_FB_DST_VOP_SUB = 8,
    E_MI_FB_DST_FRC = 9,
    E_MI_FB_DST_VE = 10,
    E_MI_FB_DST_BYPASS = 11,
    E_MI_FB_DST_OP1 = 12,
    E_MI_FB_DST_MIXER2OP1 = 13,
    E_MI_FB_DST_DIP = 14,
    E_MI_FB_DST_COPScaling = 15,
    E_MI_FB_DST_OP_DUAL_RATE = 16,
    E_MI_FB_DST_INVALID = 17,
}MI_FB_DstDisplayPlane_e
```

➤ 成员

成员名称	描述
E_MI_FB_DST_OP0	OP0 path
E_MI_FB_DST_OP1	OP1 path

目前只支持 E_MI_FB_DST_OP0/E_MI_FB_DST_OP1

➤ 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#)

3.2.7 MI_FB_DisplayLayerAttrMaskBit_e

➤ 说明

指定[MI_FB_DisplayLayerAttr_t](#)中需要更新的属性

➤ 定义

```
typedef enum
{
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS = 0x1,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE = 0x2,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_BUFFER_SIZE = 0x4,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SCREEN_SIZE = 0x8,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_PREMUL = 0x10,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_COLOR_FMB = 0x20,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_OUTPUT_COLORSPACE = 0x40,
    E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_DST_DISP = 0x80,
}MI_FB_DisplayLayerAttrMaskBit_e
```

➤ 成员

成员名称	描述
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_POS	更新显示位置
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SIZE	更新显示范围
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_BUFFER_SIZE	更新buff size
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_SCREEN_SIZE	更新屏幕分辨率
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_PREMUL	更新premultiply alpha
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_COLOR_FMB	更新pixel format
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_OUTPUT_COLORSPACE	更新输出颜色空间
E_MI_FB_DISPLAYLAYER_ATTR_MASK_DISP_DST_DISP	更新叠加层输出到display chain的目的模组

➤ 相关数据类型及接口

[MI_FB_DisplayLayerAttr_t](#)

3.2.8 MI_FB_CursorAttrMaskBit_e

➤ 说明

指定[MI_FB_CursorAttr_t](#)中需要更新的属性

➤ 定义

```
typedef enum
{
    E_MI_FB_CURSOR_ATTR_MASK_ICON = 0x1,
    E_MI_FB_CURSOR_ATTR_MASK_POS = 0x2,
    E_MI_FB_CURSOR_ATTR_MASK_ALPHA = 0x4,
    E_MI_FB_CURSOR_ATTR_MASK_SHOW = 0x8,
    E_MI_FB_CURSOR_ATTR_MASK_HIDE = 0x10,
```

```
E_MI_FB_CURSOR_ATTR_MASK_COLORKEY = 0x20,
E_MI_FB_CURSOR_ATTR_MASK = 0x3F,
}MI_FB_CursorAttrMaskBit_e
```

➤ 成员

成员名称	描述
E_MI_FB_CURSOR_ATTR_MASK_ICON	鼠标图标发生变化
E_MI_FB_CURSOR_ATTR_MASK_POS	鼠标位置发生变化
E_MI_FB_CURSOR_ATTR_MASK_ALPHA	Alpha发生变化
E_MI_FB_CURSOR_ATTR_MASK_SHOW	显示鼠标
E_MI_FB_CURSOR_ATTR_MASK_HIDE	隐藏鼠标
E_MI_FB_CURSOR_ATTR_MASK_COLORKEY	Color key发生变化
E_MI_FB_CURSOR_ATTR_MASK	用于标识鼠标所有可设置属性

➤ 相关数据类型及接口

[MI_FB_CursorAttr_t](#)

3.2.9 MI_FB_DisplayLayerAttr_t

➤ 说明

描述叠加层属性信息

➤ 定义

```
typedef struct
{
    MI_U32 u32XPos;        /**the x pos of origin point in screen*/
    MI_U32 u32YPos;        /**the y pos of origin point in screen*/
    MI_U32 u32DstWidth;     /**display buffer dest with in screen*/
    MI_U32 u32DstHeight;    /**display buffer dest hight in screen*/
    MI_U32 u32DisplayWidth; /**the width of display buf in fb */
    MI_U32 u32DisplayHeight; /**the height of display buf in fb. */
    MI_U32 u32ScreenWidth;  /**the width of screen */
    MI_U32 u32ScreenHeight; /** the height of screen */
    MI_BOOL bPreMul;        /**the data drawn in buffer whether is premultiply alpha or not*/
    MI_FB_ColorFmt_e eFbColorFmt; /**the color format of framebuffer*/
    MI_FB_OutputClrSpace_e eFbOutputColorSpace; /**output color space*/
    MI_FB_DstDisplayPlane_e eFbDestDisplayPlane; /**destination displayplane*/
    MI_U32 u32DisplayInfoMask; /** display attribute modify mask*/
}MI_FB_DisplayLayerAttr_t;
```

➤ 成员

成员名称	描述	支持情况
u32XPos	图层在屏幕上的原点横坐标。	支持
u32YPos	图层在屏幕上的原点纵坐标。	支持
u32DstWidth	图层在屏幕上的显示宽度	支持
u32DstHeight	图层在屏幕上的显示高度	支持
u32DisplayWidth	显存分辨率的宽。	支持
u32DisplayHeight	显存分辨率的高。	支持
u32ScreenWidth	屏幕显示分辨率的宽。	支持
u32ScreenHeight	屏幕显示分辨率的高。	支持
bPreMul	FB 中的数据是否为预乘数据。	支持
eFbColorFmt	FB的像素格式。	支持
u32SetAttrMask	设置图层信息时参数修改掩码位。	支持
eFbOutputColorSpace	设置叠加层输出的Color space	支持
eFbDestDisplayPlane	设置叠加层输出到display chain的目的模组。默认为E_MI_FB_DST_OP0。	支持

➤ 相关数据类型及接口

[MI_FB_GetDisplayLayerAttr](#)

[MI_FB_SetDisplayLayerAttr](#)

3.2.10 MI_FB_GlobalAlpha_t

➤ 说明

Alpha信息结构体

➤ 定义

```
typedef struct
{
    MI_BOOL bAlphaEnable;    /* alpha enable flag */
    MI_BOOL bAlphaChannel;   /* alpha channel enable flag */
    MI_BOOL bAlphaInvert;
    MI_U8 u8Alpha0;          /* alpha0 value */
    MI_U8 u8Alpha1;          /* alpha1 value */
    MI_U8 u8GlobalAlpha;     /* global alpha value */
    MI_U8 u8Reserved;
} MI_FB_GlobalAlpha_t;
```

➤ 成员

成员名称	描述	支持情况
bAlphaEnable	Alpha 叠加使能，默认为 1。	支持
bAlphaChannel	Alpha 通道使能，默认为 0。	支持
bAlphaInvert	Alpha invert flag	支持

成员名称	描述	支持情况
u8Alpha0	Alpha0 值，范围 0~255，默认为 255。在 RGB1:5:5:5 格式下，当最高位为 0 时，选择该值作为 Alpha 叠加的 Alpha 值。	支持
u8Alpha1	Alpha1 值，范围 0~255，默认为 255。在 RGB1:5:5:5 格式下，当最高位为 1 时，选择该值作为 Alpha 叠加的 Alpha 值。	支持
u8GlobalAlpha	全局 Alpha 值，范围为 0~255，默认为 255。在 Alpha 通道使能时起作用。	支持

➤ 相关数据类型及接口

[MI_FB_GetGlobalAlpha](#)

[MI_FB_SetGlobalAlpha](#)

3.2.11 MI_FB_ColorKey_t

➤ 说明

描述color key信息结构体

➤ 定义

```
typedef struct
{
    MI_BOOL bKeyEnable;
    MI_U8 u8Red;
    MI_U8 u8Green;
    MI_U8 u8Blue;
} MI_FB_ColorKey_t
```

➤ 成员

成员名称	描述	支持情况
bKeyEnable	Colorkey 是否使能标识。 TRUE: 使能; FALSE: 不使能。	支持
u8Red	Colorkey的R分量数值	支持
u8Green	Colorkey的G分量数值	支持
u8Blue	Colorkey的B分量数值	支持

➤ 相关数据类型及接口

[MI_FB_GetColorKey](#)

[MI_FB_SetColorKey](#)

3.2.12 MI_FB_CursorImage_t

➤ 说明

描述鼠标图标数据信息

➤ 定义

```
typedef struct
{
    MI_U32 u32Width;    /**width, unit pixel*/
    MI_U32 u32Height;   /**Height, unit pixel*/
    MI_U32 u32Pitch;    /**Pitch, unit pixel*/
    MI_FB_ColorFmt_e eColorFmt; /**Color format*/
    const char *pData;
}MI_FB_CursorImage_t;
```

➤ 成员

成员名称	描述	支持情况
u32Width	图标宽度	支持
u32Height	图标高度	支持
u32Pitch	pitch	支持
eColorFmt	图标数据像素格式	支持
pData	图标数据	支持

➤ 相关数据类型及接口

[MI_FB_CursorAttr_t](#)

3.2.13 MI_FB_CursorAttr_t

➤ 说明

描述鼠标图层信息

➤ 定义

```
typedef struct
{
    MI_U32 u32XPos;
    MI_U32 u32YPos;
    MI_U32 u32HotSpotX;
    MI_U32 u32HotSpotY;
    MI_FB_GlobalAlpha_t stAlpha;
    MI_FB_ColorKey_t stClrKey;
    MI_BOOL bShown;
    MI_FB_CursorImage_t stCursorImageInfo;
    MI_U16 u16CursorInfoMask;
}MI_FB_CursorAttr_t;
```

➤ 成员

成员名称	描述	支持情况
u32XPos	鼠标位置横坐标	支持
u32YPos	鼠标位置纵坐标	支持
u32HotSpotX	鼠标热点横坐标	支持
u32HotSpotY	鼠标热点纵坐标	支持
stAlpha	MI_FB_GlobalAlpha_t结构体,鼠标图层alpha信息	支持
stColorKey	MI_FB_ColorKey_t结构体,鼠标图层colorkey信息	支持
bShown	鼠标是否可见 TRUE: 表示可见 FALSE: 表示不可见	支持
MI_FB_CursorImage_t	图标数据信息。	支持
u16CursorInfoMask	设置鼠标图层参数修改掩码位	支持

➤ 相关数据类型及接口

[MI_FB_GetCursorAttr](#)

[MI_FB_SetCursorAttr](#)

3.2.14 MI_FB_Rectangle_t

➤ 说明

描述叠加层的显示区域

➤ 定义

```
typedef struct
{
    MI_U32 u32XPos;
    MI_U32 u32YPos;
    MI_U32 u32Width;
    MI_U32 u32Height;
}MI_FB_Rectangle_t;
```

➤ 成员

成员名称	描述	支持情况
u32XPos	左上角x坐标	支持
u32YPos	左上角y坐标	支持
u32Width	宽度	支持
u32Height	高度	支持

➤ 相关数据类型及接口

[MI_FB_GetScreenLocation](#)

[MI_FB_SetScreenLocation](#)

3.2.15 MI_FB_Cmap_t

➤ 说明

描述信息和标准定义相同，参考struct [fb_cmap_user](#)

➤ 相关数据类型及接口

[MI_FB_SetCmap](#)

[MI_FB_GetCmap](#)

4. 错误码

错误代码	宏定义	描述
0x0	MI_SUCCESS	succeeded
0xA00B2011	MI_ERR_FB_INVALID_PARAM	非法参数
0xA00B2006	MI_ERR_FB_NULL_PTR	空指针异常
0xA00B2001	MI_ERR_FB_INVALID_DEVID	无效设备号
0xA00B2018	MI_ERR_FB_DRV_FAILED	FB 驱动返回失败
0xA00B2015	MI_ERR_FB_NOT_INIT	FB 未初始化
0xA00B2008	MI_ERR_FB_NOT_SUPPORT	不支持操作

5. PROCFS 介绍

5.1.

cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_fb0
```

```
#####OnDumpOSDAttr#####
Framebuffer id = MStar FB0
xres=1280, yres=720
xres_virtual=1280, yres_virtual=1440
xoffset=0,yoffset=0
fix.line_length=0xa00 Bytes
fix.smem_start=0xf800000
Memory Size=0x400000 Bytes
Gop ID=0
Gwin ID=0
Open Count=1
Visible State=0
MIU Sel=0
ColorSpace=YUV
ColorFomrmat=ARGB1555
StretchWindow Pos[0,0]
StretchWindow Src[1280,720],StretchWindow Dst[1920,1080]
Gwin Pos[0,0]
Gwin Size[1280,720]
Gwin PhyAddr=0x0
ColorKey Enable=0
ColorKey Val=[0,0,0]
Enable Alpha Blend=1
Enalbe Multi Alpha=0
Global Alpha Val=0x0
Alpha0=0x0,Alpha1=0x0,(Only for ARGB1555)
GOP Hstart=192
Current TimingWidth=1920,TimingWidth=1080,hstar=192
#####end#####

#####OnDumpHwcursorAttr#####
Cursor Gop ID=4
Cursor Gwin ID=6
Cursor MIU Sel=0
Cursor PhyAddr=0x166000
Cursor ColorFmt=ARGB8888
Cursor Icon Width=44,Height=56
Cursor HotSpot[18,9]
Cursor request pos[712,207]
Cursor Visible=1
Cursor Gwin Pos[694,198]
```

```
Cursor Gwin Size[44,56]
Cursor Gwin Pitch=0x200 Bytes
Curosr StretchWindow pos[0,0]
Cursor StretchWin Src[1280,720],Dst[1920,1080]
Cursor ColorKey Enable=0
Cursor ColorKey Value=[0xff,0xff,0xff]
#####OnDumpHwcursorAttr#####
```

➤ 调试信息分析

记录当前 Fbdev && Hwcursor 的使用状况以及 OSD 图层属性、OSD device 属性、Hwcursor 属性，可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

参数		描述
OSD info	Framebuffer id	Framebuffer 设备 ID
	xres	可见分辨率宽度
	yres	可见分辨率宽高度
	xres_virtual	虚拟分辨率的宽度
	yres_virtual	虚拟分辨率的高度
	xoffset	可见分辨率到虚拟分辨率的 x 偏移量
	yoffset	可见分辨率到虚拟分辨率的 y 偏移量
	fix.line_length	虚拟分辨率每行的字节数
	fix.smem_start	帧缓冲内起始地址
	Memory Size	帧缓冲内存长度，单位字节数
OSD device info	Gop ID	图层硬件 ID
	Gwin ID	图层硬件显示窗口 ID
	Open Count	该图形层打开次数。在用户调用 open() 时增 1；调用 close() 时减 1
	Visible State	该图层显示状态。 取值:{1 表示可见, 0 表示不可见} 用户显示调用 FBIOSET_SHOW 去设置图层可见状态。
	MIU Sel	保存帧缓冲数据的物理内存编号。 对于 MSR620 设备该值为 0
	ColorSpace	该图形层输出的颜色空间。 取值:{YUV 表示输出 YUV color space; RGB 表示输出 RGB color space }。默认为 YUV
	ColorFomrmat	图形层格式 取值:{ARGB1555, ARGB4444, ARGB8888, RGB565}。 用户设置可变屏幕信息中的格式项后更新
	StretchWindow Pos	图形层在显示设备上的起始位置。 单位: 像素

参数	描述
	默认为 0,用户可调用 FBIOSET_SCREEN_LOCATION 去设置
StretchWindow Src	图形层原始分辨率的宽度和高度。 单位:像素 通过硬件放大功能放大到当前 timing。用户设置可变屏幕信息中的可见分辨率后更新
StretchWindow Dst	图形层在显示设备上的宽度和高度。 默认设置为输出 timing 大小。可以通过 FBIOSET_SCREEN_LOCATION 去设置
Gwin Pos	图形层显示窗口的起始位置 单位:像素 对于 OSD 图层来说,该值总是为(0,0)
Gwin Size	图形层显示窗口的宽度、高度。 单位:像素 对于 OSD 图层来说,该值总是被设置为可见分辨率的宽度、高度。用户设置可变屏幕信息中的可见分辨率后更新。
Gwin PhyAddr	图形层当前显示数据的物理地址
ColorKey Enable	图形层 ColorKey 是否使能 取值:{0 表示 Disable ColorKey; 1 表示 Enable ColorKey}用户设置可通过 FBIOSET_COLORKEY 去设置
ColorKey Val	图形层 ColorKey 的 RED, GREEN, BLUE 通道数值 取值:每个通道的取值范围从 0-255。ColorKey val 的数值用 RGB888 格式表示 用户可通过 FBIOSET_COLORKEY 去设置后更新
Enable Alpha Blend	图形层 alpha 是否使能。 取值:{0 表示否, 1 表示是},默认为 1。 用户可通过 FBIOSET_GLOBAL_ALPHA 去设置后更新。 该项关闭,则像素 alpha 配置不生效。该项开启且 Enable Multi Alpha 关闭,仅像素 alpha 生效(对于 ARGB1555 格式,Alpha bit 为 1 的像素使用 Global Alpha 作为像素 alpha,Alpha bit 为 0 的像素使用 0xff 作为像素 alpha)。当该项目开启且 Enable Multi Alpha 生效时,则像素 alpha 和 Global Alpha 都生效
Enable Multi Alpha	全局 alpha 是否生效开关 取值{0 时, 1 否}默认值为 0 Enable Multi Alpha 使能, Global Alpha 便生效
Global Alpha Val	全局 Alpha
Alpha0	ARGB1555 格式时,当最高位为 0 时,选择 Alpha0 作为 Alpha 混合的 Alpha 数值 取值: 0~255 默认为 0

参数		描述
	Alpha1	ARGB1555 格式时，当最高位为 1 时，选择 Alpha1 作为 Alpha 混合的 Alpha 数值 取值: 0~255 默认为 0
	GOP Hstart	图层硬件向对于显示窗口的偏移。 取值:该值和当前输出 timing 相关，当输出 timing 发生变化时，通过 FBIOSET_DISPLAYLAYER_ATTRIBUTES 并将输入参数 MI_FB_DisplayLayerAttr_s 的 u32SetAttrMask 数值包含 E_MI_FB_DISPLAYLAYER_ATTR_MASK_SCREEN_SIZE mask 位来通知图层驱动输出 timing 发生变化。图层驱动会根据当前 timing 设置相应的 hstart 数值
	Current TimingWidth,	当前输出 timing 的宽度、高度以及当前 timing 下应该给图形层硬件设置的图像对于显示窗口的偏移。 该项与 GOP Hstart 配合使用，用于检查设置给硬件图层的 hstart 数值是否和当前 timing 一致。
	Current TimingHeight,	
	hstar	
Hwcursor info	Cursor Gop ID	鼠标图层硬件 ID
	Cursor Gwin ID	鼠标图层硬件显示窗口 ID
	Cursor MIU Sel	用于保存鼠标图标数据的物理内存编号。 对于 MSR620 设备该值为 0
	Cursor PhyAddr	鼠标层当前显示数据的物理地址
	Cursor ColorFmt	鼠标图层格式 值:{ARGB1555,ARGB4444,ARGB8888,RGB565}。 用户可通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor Icon Width	鼠标图标的宽度、高度。 单位: 像素 用户可通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor HotSpot	鼠标图标的热点信息 单位:像素 热点信息通常和鼠标图标相关，用户可通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor request pos	鼠标图标的位置信息 单位: 像素 用户通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor Visible	鼠标图层是否可见 取值:{0,表示不可见, 1 表示可见} 用户通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor Gwin Pos	鼠标图层显示窗口的起始位置

参数		描述
		单位:像素
	Cursor Gwin Pitch	鼠标图层每行的字节数。 对于鼠标图层来说,该值固定为 0x200
	Curosr StretchWindow pos	鼠标图层在显示设备上的起始位置。 单位:像素
	Cursor StretchWin Src	鼠标图层原始大小 单位:像素。该值通常和 OSD 的显示分辨率一致。
	Cursor StretchWin Dst	鼠标图层在显示设备上的宽度、高度 该值和输出 timing 一致。
	Cursor ColorKey Enable	鼠标图层 ColorKey 是否使能 取值:{0 表示 Disable ColorKey; 1 表示 Enable ColorKey}。 用户通过 FBIOSET_CURSOR_ATTRIBUTE 设置
	Cursor ColorKey Value	鼠标图层 ColorKey 的 RED, GREEN, BLUE 通道数值 取值:每个通道的取值范围从 0-255。ColorKey val 的数值用 RGB888 格式表示 用户可通过 FBIOSET_CURSOR_ATTRIBUTE 去设置后更新

5.2.

echo

功能	打开或关闭指定的图层
命令	echo GUI_SHOW [FbID] [Status] > /proc/mi_modules/mi_fb0
参数说明	[FbID] UI 层所在的 fb 设备 index
	[Status] on 打开 off 关闭
举例	echo GUI_SHOW 0 on > /proc/mi_modules/mi_fb0
	显示图层 fb0
	echo GUI_SHOW 0 off > /proc/mi_modules/mi_fb0
	显示图层 fb0

功能	打开或关闭硬件鼠标
命令	echo CURSOR_SHOW [FbID] [Status] > /proc/mi_modules/mi_fb0
参数说明	[FbID] 鼠标层所在的 fb 设备 index
	[Status] on 打开 off 关闭
举例	echo CURSOR_SHOW 0 on > /proc/mi_modules/mi_fb0
	显示鼠标图标
	echo CURSOR_SHOW 0 off > /proc/mi_modules/mi_fb0

	关闭鼠标图标
--	--------

功能	为指定的图层设置 Colorkey
命令	echo GUI_SET_CLRKEY [FbID] [Red] [Green] [Blue] > /proc/mi_modules/mi_fb0
参数说明	[FbID] UI 层所在的 fb 设备 index
	[Red] 红色分量 以 16 进制,ARGB888 格式表示
	[Green] 绿色分量 以 16 进制,ARGB888 格式表示
	[Blue] 蓝色分量 以 16 进制,ARGB888 格式表示
举例	echo GUI_SET_CLRKEY 0 ff 00 00 > /proc/mi_modules/mi_fb0 设置 fb0 的 Colorkey 为红色

功能	Dump 当前正在显示的鼠标图标
命令	echo CURSOR_DUMP [FbID] [Path] >/proc/mi_modules/mi_fb0
参数说明	[FbID] 鼠标层所在的 fb 设备 index
	[Path] dump 的路径
举例	echo CURSOR_DUMP 0 /mnt > /proc/mi_modules/mi_fb0 dump cursor Icon 到/mnt 路径，文件名为 CursorData.raw，用 7yuv 打开，宽高设置为 128，格式按照 cursor icon 的格式设置即可显示 Rawdata

功能	分别为 ARGB1555 格式的 alpha bit0,alpha bi1 设置 pixel alpha 数值。因为目前 msr620 不支持这个 feature，暂时未实现
命令	echo GUI_SETALPHA_ARGB1555 [FbID] [Alpha0] [Alpha1]> /proc/mi_modules/mi_fb0
参数说明	[FbID] UI 层所在的 fb 设备 index
	[Alpha0] alpha0 的数值
	[Alpha1] alpha1 的数值
举例	无